



LABFluid

Laboratório de Dinâmica
dos Fluidos Computacional

Samuel Kutz Paranhos

Universidade Federal do Paraná (UFPR)

Bacharelado em Matemática Industrial

Orientador: Prof. Roberto Ribeiro

Métodos de Scientific Machine Learning para o Sistema Não Linear de Boussinesq

Sumário

- 1 Introdução
- 2 Revisão da Literatura
- 3 Fundamentação Matemática

- 4 Metodologia
- 5 Resultados
- 6 Conclusão

Introdução



Onda quase solitária gerada em um canal de águas rasas (Pizzo 2016).

- A matemática é uma das principais linguagens com que descrevemos fenômenos como este, e as equações diferenciais parciais (EDPs), um de seus dialetos;



Onda quase solitária gerada em um canal de águas rasas (Pizzo 2016).

- ▶ A matemática é uma das principais linguagens com que descrevemos fenômenos como este, e as equações diferenciais parciais (EDPs), um de seus dialetos;
- ▶ Nem sempre essas equações possuem formas fechadas para soluções, isso quando possuem; é para isso que usamos métodos numéricos;



Onda quase solitária gerada em um canal de águas rasas (Pizzo 2016).

- ▶ A matemática é uma das principais linguagens com que descrevemos fenômenos como este, e as equações diferenciais parciais (EDPs), um de seus dialetos;
- ▶ Nem sempre essas equações possuem formas fechadas para soluções, isso quando possuem; é para isso que usamos métodos numéricos;
- ▶ Este trabalho busca aplicar métodos de Scientific Machine Learning na aproximação da solução de um sistema de EDPs, o sistema de Boussinesq.

Em resumo, estudaremos métodos de Scientific Machine Learning (SciML), uma subárea de Machine Learning (ML) na interseção com a Computação Científica, termo consolidado por Baker et al. (2019) e revisado por Karniadakis et al. (2021) e Dietrich e Schilders (2025).

- ▶ A ascensão recente de ML veio acompanhada de todo um ecossistema de ferramentas: bibliotecas, otimizadores e aceleradores que receberam investimento nas últimas décadas;

Em resumo, estudaremos métodos de Scientific Machine Learning (SciML), uma subárea de Machine Learning (ML) na interseção com a Computação Científica, termo consolidado por Baker et al. (2019) e revisado por Karniadakis et al. (2021) e Dietrich e Schilders (2025).

- ▶ A ascensão recente de ML veio acompanhada de todo um ecossistema de ferramentas: bibliotecas, otimizadores e aceleradores que receberam investimento nas últimas décadas;
- ▶ Um caminho natural é aproveitar esse ecossistema na solução numérica de EDPs, e é isso que os métodos deste trabalho fazem.

Aqui, estudaremos o Sistema de Boussinesq na forma paramétrica de Martins (2023), que parte do modelo com topografia h e pressão P_0 na superfície, em $x \in [-L, L]$, a saber:

$$\begin{cases} \eta_t + ((1 - \alpha h + \alpha \eta) u)_x = h_t, \\ u_t + \eta_x + \alpha u u_x - \frac{\beta}{3} u_{xxt} = -\frac{\beta}{2} h_{xtt} - \alpha (P_0)_x \end{cases} \xrightarrow[(P_0)_x=0]{h=0} \begin{cases} \eta_t + u_x + \alpha (\eta u)_x = 0, \\ u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \alpha u u_x = 0. \end{cases}$$

- $\eta(x, t)$: elevação da superfície da onda;

Aqui, estudaremos o Sistema de Boussinesq na forma paramétrica de Martins (2023), que parte do modelo com topografia h e pressão P_0 na superfície, em $x \in [-L, L]$, a saber:

$$\begin{cases} \eta_t + ((1 - \alpha h + \alpha \eta) u)_x = h_t, \\ u_t + \eta_x + \alpha u u_x - \frac{\beta}{3} u_{xxt} = -\frac{\beta}{2} h_{xtt} - \alpha (P_0)_x \end{cases} \xrightarrow[(P_0)_x=0]{h=0} \begin{cases} \eta_t + u_x + \alpha (\eta u)_x = 0, \\ u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \alpha u u_x = 0. \end{cases}$$

- ▶ $\eta(x, t)$: elevação da superfície da onda;
- ▶ $u(x, t)$: média vertical da velocidade horizontal da onda;

Aqui, estudaremos o Sistema de Boussinesq na forma paramétrica de Martins (2023), que parte do modelo com topografia h e pressão P_0 na superfície, em $x \in [-L, L]$, a saber:

$$\begin{cases} \eta_t + ((1 - \alpha h + \alpha \eta) u)_x = h_t, \\ u_t + \eta_x + \alpha u u_x - \frac{\beta}{3} u_{xxt} = -\frac{\beta}{2} h_{xtt} - \alpha (P_0)_x \end{cases} \xrightarrow[(P_0)_x=0]{h=0} \begin{cases} \eta_t + u_x + \alpha (\eta u)_x = 0, \\ u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \alpha u u_x = 0. \end{cases}$$

- ▶ $\eta(x, t)$: elevação da superfície da onda;
- ▶ $u(x, t)$: média vertical da velocidade horizontal da onda;
- ▶ $\alpha \geq 0$ controla as não linearidades $(\eta u)_x$ e $u u_x$;

Aqui, estudaremos o Sistema de Boussinesq na forma paramétrica de Martins (2023), que parte do modelo com topografia h e pressão P_0 na superfície, em $x \in [-L, L]$, a saber:

$$\begin{cases} \eta_t + ((1 - \alpha h + \alpha \eta) u)_x = h_t, \\ u_t + \eta_x + \alpha u u_x - \frac{\beta}{3} u_{xxt} = -\frac{\beta}{2} h_{xtt} - \alpha (P_0)_x \end{cases} \xrightarrow[(P_0)_x=0]{h=0} \begin{cases} \eta_t + u_x + \alpha (\eta u)_x = 0, \\ u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \alpha u u_x = 0. \end{cases}$$

- ▶ $\eta(x, t)$: elevação da superfície da onda;
- ▶ $u(x, t)$: média vertical da velocidade horizontal da onda;
- ▶ $\alpha \geq 0$ controla as não linearidades $(\eta u)_x$ e $u u_x$;
- ▶ $\beta > 0$ controla a dispersão u_{xxt} ;

Aqui, estudaremos o Sistema de Boussinesq na forma paramétrica de Martins (2023), que parte do modelo com topografia h e pressão P_0 na superfície, em $x \in [-L, L]$, a saber:

$$\begin{cases} \eta_t + ((1 - \alpha h + \alpha \eta) u)_x = h_t, \\ u_t + \eta_x + \alpha u u_x - \frac{\beta}{3} u_{xxt} = -\frac{\beta}{2} h_{xxt} - \alpha (P_0)_x \end{cases} \xrightarrow[(P_0)_x=0]{h=0} \begin{cases} \eta_t + u_x + \alpha (\eta u)_x = 0, \\ u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \alpha u u_x = 0. \end{cases}$$

- ▶ $\eta(x, t)$: elevação da superfície da onda;
- ▶ $u(x, t)$: média vertical da velocidade horizontal da onda;
- ▶ $\alpha \geq 0$ controla as não linearidades $(\eta u)_x$ e $u u_x$;
- ▶ $\beta > 0$ controla a dispersão u_{xxt} ;
- ▶ Condição inicial fixada: $\eta(x, 0) = A \operatorname{sech}^2(x)$, $u(x, 0) = 0$.

Aqui, estudaremos o Sistema de Boussinesq na forma paramétrica de Martins (2023), que parte do modelo com topografia h e pressão P_0 na superfície, em $x \in [-L, L]$, a saber:

$$\begin{cases} \eta_t + ((1 - \alpha h + \alpha \eta) u)_x = h_t, \\ u_t + \eta_x + \alpha u u_x - \frac{\beta}{3} u_{xxt} = -\frac{\beta}{2} h_{xtt} - \alpha (P_0)_x \end{cases} \xrightarrow[(P_0)_x=0]{h=0} \begin{cases} \eta_t + u_x + \alpha (\eta u)_x = 0, \\ u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \alpha u u_x = 0. \end{cases}$$

- ▶ $\eta(x, t)$: elevação da superfície da onda;
- ▶ $u(x, t)$: média vertical da velocidade horizontal da onda;
- ▶ $\alpha \geq 0$ controla as não linearidades $(\eta u)_x$ e $u u_x$;
- ▶ $\beta > 0$ controla a dispersão u_{xxt} ;
- ▶ Condição inicial fixada: $\eta(x, 0) = A \operatorname{sech}^2(x)$, $u(x, 0) = 0$.
- ▶ Formato que utilizamos veio de Martins (2023), que também fornece meios de resolver numericamente por método pseudoespectral, que será nossa solução de referência.

Avaliar o desempenho de 4 arquiteturas de SciML na aproximação da solução do sistema de Boussinesq, a saber:

	Dados puros	Dados e física	Física pura
Redes Neurais (NN)	MLP	PINN	PINN
Operadores Neurais (NO)	FNO	PINO	PINO

- 1 Qual é melhor: os dados, a física ou os dois juntos? O resíduo da EDP ajuda?

¹Viés espectral é a tendência de redes neurais aprenderem primeiro componentes de baixa frequência, e só depois os de alta frequência, durante a otimização.

Avaliar o desempenho de 4 arquiteturas de SciML na aproximação da solução do sistema de Boussinesq, a saber:

	Dados puros	Dados e física	Física pura
Redes Neurais (NN)	MLP	PINN	PINN
Operadores Neurais (NO)	FNO	PINO	PINO

- 1 Qual é melhor: os dados, a física ou os dois juntos? O resíduo da EDP ajuda?
- 2 Há viés espectral evidente nesses métodos? ¹

¹Viés espectral é a tendência de redes neurais aprenderem primeiro componentes de baixa frequência, e só depois os de alta frequência, durante a otimização.

Avaliar o desempenho de 4 arquiteturas de SciML na aproximação da solução do sistema de Boussinesq, a saber:

	Dados puros	Dados e física	Física pura
Redes Neurais (NN)	MLP	PINN	PINN
Operadores Neurais (NO)	FNO	PINO	PINO

- 1 Qual é melhor: os dados, a física ou os dois juntos? O resíduo da EDP ajuda?
- 2 Há viés espectral evidente nesses métodos? ¹
- 3 Uma análise via Neural Tangent Kernel (NTK) consegue descrever bem o viés espectral?

¹Viés espectral é a tendência de redes neurais aprenderem primeiro componentes de baixa frequência, e só depois os de alta frequência, durante a otimização.

Avaliar o desempenho de 4 arquiteturas de SciML na aproximação da solução do sistema de Boussinesq, a saber:

	Dados puros	Dados e física	Física pura
Redes Neurais (NN)	MLP	PINN	PINN
Operadores Neurais (NO)	FNO	PINO	PINO

- 1 Qual é melhor: os dados, a física ou os dois juntos? O resíduo da EDP ajuda?
- 2 Há viés espectral evidente nesses métodos? ¹
- 3 Uma análise via Neural Tangent Kernel (NTK) consegue descrever bem o viés espectral?
- 4 Para Operadores Neurais, há realmente invariância perante discretização?

¹Viés espectral é a tendência de redes neurais aprenderem primeiro componentes de baixa frequência, e só depois os de alta frequência, durante a otimização.

Revisão da Literatura

Uma Physics-Informed Neural Network (PINN) (Raissi et al. 2019) adiciona o resíduo da EDP na função objetivo, construindo uma rede que aproxima a solução da equação/respeita a lei física.

Trabalho	Equação	Forma resolvida
Bihlo e Popovych (2022)	Águas rasas na esfera	$(\eta - h)_t + \nabla \cdot ((\eta - h) \mathbf{u}) = 0,$ $\mathbf{u}_t + (\mathbf{u} \cdot \nabla) \mathbf{u} + f \mathbf{k} \times \mathbf{u} + g \nabla \eta = 0$
Jagtap et al. (2022)	Serre–Green–Naghdi (1D)	$(h + \eta)_t + ((h + \eta) u)_x = 0,$ $u_t + g \eta_x + u u_x - \frac{1}{h + \eta} [(h + \eta)^2 (\frac{1}{3} P + \frac{1}{2} Q)]_x$ $+ (\frac{1}{2} P + Q) h_x = 0$
Liu et al. (2023)	Boussinesq (2 + 1)-dimensional	—
Su et al. (2024)	Boussinesq e Camassa–Holm	$\eta_{tt} - \eta_{xx} - 3(\eta^2)_{xx} - \eta_{xxxx} = 0,$ $u_t + 2\kappa u_x - u_{xxt} + 3u u_x - 2u_x u_{xx} - u u_{xxx} = 0$

Note que não há trabalhos de SciML para o sistema de Boussinesq na forma paramétrica de Martins (2023), que é o foco deste trabalho.

Outro braço de métodos dentro de SciML, buscando aproximar o operador que leva o parâmetro da EDP à solução (operador entre espaços de função).

Trabalho	Contribuição
Lu et al. (2021)	DeepONet: primeira arquitetura profunda para operadores
Li et al. (2021)	FNO: kernel em Fourier, independente da discretização
Gupta et al. (2021)	bases alternativas, como wavelets
Li et al. (2023a)	PINO: resíduo da EDP no operador

- Escala geofísica: oceano regional e previsão atmosférica global (Bonev et al. 2023; Uchida et al. 2025; Yu et al. 2025);

Outro braço de métodos dentro de SciML, buscando aproximar o operador que leva o parâmetro da EDP à solução (operador entre espaços de função).

Trabalho	Contribuição
Lu et al. (2021)	DeepONet: primeira arquitetura profunda para operadores
Li et al. (2021)	FNO: kernel em Fourier, independente da discretização
Gupta et al. (2021)	bases alternativas, como wavelets
Li et al. (2023a)	PINO: resíduo da EDP no operador

- ▶ Escala geofísica: oceano regional e previsão atmosférica global (Bonev et al. 2023; Uchida et al. 2025; Yu et al. 2025);
- ▶ Independência da discretização (zero-shot superresolution), possibilidade de avaliar em uma resolução maior do que a treinada (Li et al. 2021; Li et al. 2023a).

Outro braço de métodos dentro de SciML, buscando aproximar o operador que leva o parâmetro da EDP à solução (operador entre espaços de função).

Trabalho	Contribuição
Lu et al. (2021)	DeepONet: primeira arquitetura profunda para operadores
Li et al. (2021)	FNO: kernel em Fourier, independente da discretização
Gupta et al. (2021)	bases alternativas, como wavelets
Li et al. (2023a)	PINO: resíduo da EDP no operador

- ▶ Escala geofísica: oceano regional e previsão atmosférica global (Bonev et al. 2023; Uchida et al. 2025; Yu et al. 2025);
- ▶ Independência da discretização (zero-shot superresolution), possibilidade de avaliar em uma resolução maior do que a treinada (Li et al. 2021; Li et al. 2023a).
- ▶ Também não há trabalhos aplicando para o sistema de Boussinesq

Viés espectral e o Neural Tangent Kernel

Uma característica presente ao utilizar métodos de SciML para a solução de EDPs de evolução, é o **viés espectral**, que consiste em no fato do método priorizar aproximar as baixas frequências da solução e necessitar de muito mais iterações para as altas.

- ▶ O viés espectral foi inicialmente relatado por Rahaman et al. (2019) e Xu et al. (2020), apresentado para PINNs por Wang et al. (2022) e para operadores neurais Li et al. (2023a) e Khodakarami et al. (2026);

i O que fizemos

Usamos o framework de Neural Tangent Kernel (NTK) (Jacot et al. 2018) para isolar o viés espectral para uma MLP simples, encontrando estimativas do treinamento (válidas sob certas hipóteses) (Chizat et al. 2019).

Viés espectral e o Neural Tangent Kernel

Uma característica presente ao utilizar métodos de SciML para a solução de EDPs de evolução, é o **viés espectral**, que consiste em no fato do método priorizar aproximar as baixas frequências da solução e necessitar de muito mais iterações para as altas.

- ▶ O viés espectral foi inicialmente relatado por Rahaman et al. (2019) e Xu et al. (2020), apresentado para PINNs por Wang et al. (2022) e para operadores neurais Li et al. (2023a) e Khodakarami et al. (2026);
- ▶ Explicações teóricas que encontramos são restritas a MLPs específicas, como uma rede de duas camadas com ReLU (Arora et al. 2019; Cao et al. 2021; Basri et al. 2019).

i O que fizemos

Usamos o framework de Neural Tangent Kernel (NTK) (Jacot et al. 2018) para isolar o viés espectral para uma MLP simples, encontrando estimativas do treinamento (válidas sob certas hipóteses) (Chizat et al. 2019).

Fundamentação Matemática

Formulação Matemática de Redes Neurais

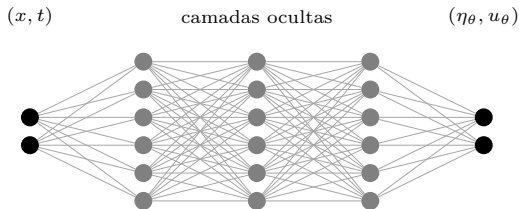
Composição de transformações afins separadas por ativações não lineares, ajustada por erro quadrático médio.

$$h^{(0)} = x,$$

$$h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right),$$

$$u_{\theta}(x) = W^{(L)}h^{(L-1)} + b^{(L)}.$$

► $\theta = \{W^{(l)}, b^{(l)}\}$: parâmetros treináveis;



Formulação Matemática de Redes Neurais

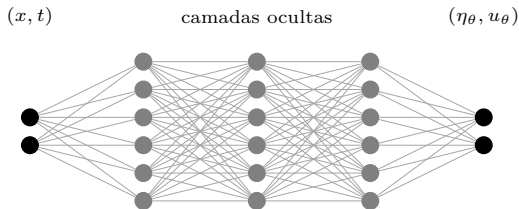
Composição de transformações afins separadas por ativações não lineares, ajustada por erro quadrático médio.

$$h^{(0)} = x,$$

$$h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right),$$

$$u_{\theta}(x) = W^{(L)}h^{(L-1)} + b^{(L)}.$$

- ▶ $\theta = \{W^{(l)}, b^{(l)}\}$: parâmetros treináveis;
- ▶ σ sigmoidal aplicada elemento a elemento;



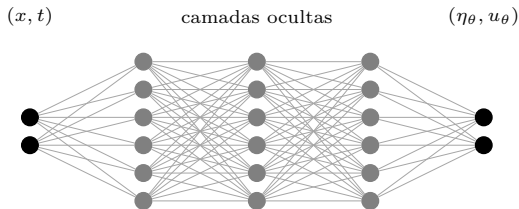
Formulação Matemática de Redes Neurais

Composição de transformações afins separadas por ativações não lineares, ajustada por erro quadrático médio.

$$h^{(0)} = x,$$

$$h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right),$$

$$u_{\theta}(x) = W^{(L)}h^{(L-1)} + b^{(L)}.$$



- ▶ $\theta = \{W^{(l)}, b^{(l)}\}$: parâmetros treináveis;
- ▶ σ sigmoidal aplicada elemento a elemento;
- ▶ Função de perda padrão contra um conjunto de dados $\mathcal{S} = \{(x_i, u_i)\}_{i=1}^N$:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_i \|u_{\theta}(x_i) - u_i\|^2.$$

A PINN (Raissi et al. 2019) insere a EDP na otimização: além dos dados, a rede é forçada a satisfazer as equações.

$$\mathcal{L}_{\text{PINN}}(\theta) = \underbrace{\frac{1}{N_{\mathcal{D}}} \sum_i \|\mathcal{D}[u_{\theta}](x_i, t_i)\|^2}_{\text{resíduo da EDP}} + \underbrace{\frac{1}{N_0} \sum_i \|u_{\theta}(x_i, 0) - u_0(x_i)\|^2}_{\text{condição inicial}}.$$

Para o sistema de Boussinesq, $\mathcal{D}$ é o par de resíduos

$$\mathcal{D}_1 = (\eta_{\theta})_t + (u_{\theta})_x + \alpha (\eta_{\theta} u_{\theta})_x,$$

$$\mathcal{D}_2 = (u_{\theta})_t - \frac{\beta}{3} (u_{\theta})_{xxt} + (\eta_{\theta})_x + \alpha u_{\theta} (u_{\theta})_x.$$

- Derivadas por diferenciação automática (Linnainmaa 1970; Paszke et al. 2019); Adam (Kingma e Ba 2014);

A PINN (Raissi et al. 2019) insere a EDP na otimização: além dos dados, a rede é forçada a satisfazer as equações.

$$\mathcal{L}_{\text{PINN}}(\theta) = \underbrace{\frac{1}{N_{\mathcal{D}}} \sum_i \|\mathcal{D}[u_{\theta}](x_i, t_i)\|^2}_{\text{resíduo da EDP}} + \underbrace{\frac{1}{N_0} \sum_i \|u_{\theta}(x_i, 0) - u_0(x_i)\|^2}_{\text{condição inicial}}.$$

Para o sistema de Boussinesq, $\mathcal{D}$ é o par de resíduos

$$\mathcal{D}_1 = (\eta_{\theta})_t + (u_{\theta})_x + \alpha (\eta_{\theta} u_{\theta})_x,$$

$$\mathcal{D}_2 = (u_{\theta})_t - \frac{\beta}{3} (u_{\theta})_{xxt} + (\eta_{\theta})_x + \alpha u_{\theta} (u_{\theta})_x.$$

- ▶ Derivadas por diferenciação automática (Linnainmaa 1970; Paszke et al. 2019); Adam (Kingma e Ba 2014);
- ▶ (α, β) fixados, uma PINN aprende para só um par, mas há implementações que aprendem como apenas mais um canal de features.

Formulação Matemática de Operadores Neurais

Sejam $\mathcal{A}$ e $\mathcal{U}$ espaços de Banach de funções sobre D . O alvo é o operador solução $G^\dagger : \mathcal{A} \rightarrow \mathcal{U}$, remodelamos o problema como o mapeamento $(\alpha, \beta) \mapsto (\eta, u)$. Um operador neural o aproxima por

$$G_\theta = Q \circ \sigma(W_L + \mathcal{K}_L) \circ \cdots \circ \sigma(W_1 + \mathcal{K}_1) \circ P,$$

em que cada camada atualiza os canais por

$$v_{\ell+1}(x) = \sigma\left((W_\ell v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\right).$$

- $P : \mathbb{R}^{d_a} \rightarrow \mathbb{R}^{d_c}$: operador de elevação, leva a entrada ponto a ponto a $d_c \gg d_a$ canais;

Formulação Matemática de Operadores Neurais

Sejam $\mathcal{A}$ e $\mathcal{U}$ espaços de Banach de funções sobre D . O alvo é o operador solução $G^\dagger : \mathcal{A} \rightarrow \mathcal{U}$, remodelamos o problema como o mapeamento $(\alpha, \beta) \mapsto (\eta, u)$. Um operador neural o aproxima por

$$G_\theta = Q \circ \sigma(W_L + \mathcal{K}_L) \circ \cdots \circ \sigma(W_1 + \mathcal{K}_1) \circ P,$$

em que cada camada atualiza os canais por

$$v_{\ell+1}(x) = \sigma\left((W_\ell v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\right).$$

- ▶ $P : \mathbb{R}^{d_a} \rightarrow \mathbb{R}^{d_c}$: operador de elevação, leva a entrada ponto a ponto a $d_c \gg d_a$ canais;
- ▶ W_ℓ : mapa linear local; σ : ativação aplicada ponto a ponto;

Formulação Matemática de Operadores Neurais

Sejam $\mathcal{A}$ e $\mathcal{U}$ espaços de Banach de funções sobre D . O alvo é o operador solução $G^\dagger : \mathcal{A} \rightarrow \mathcal{U}$, remodelamos o problema como o mapeamento $(\alpha, \beta) \mapsto (\eta, u)$. Um operador neural o aproxima por

$$G_\theta = Q \circ \sigma(W_L + \mathcal{K}_L) \circ \cdots \circ \sigma(W_1 + \mathcal{K}_1) \circ P,$$

em que cada camada atualiza os canais por

$$v_{\ell+1}(x) = \sigma\left((W_\ell v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\right).$$

- ▶ $P : \mathbb{R}^{d_a} \rightarrow \mathbb{R}^{d_c}$: operador de elevação, leva a entrada ponto a ponto a $d_c \gg d_a$ canais;
- ▶ W_ℓ : mapa linear local; σ : ativação aplicada ponto a ponto;
- ▶ $\mathcal{K}_\ell$: operador integral não local, com kernel aprendido $\kappa_\ell : D \times D \rightarrow \mathbb{R}^{d_c \times d_c}$, que faz cada ponto da saída ler a função inteira:

$$(\mathcal{K}_\ell v)(x) = \int_D \kappa_\ell(x, y) v(y) dy;$$

Formulação Matemática de Operadores Neurais

Sejam $\mathcal{A}$ e $\mathcal{U}$ espaços de Banach de funções sobre D . O alvo é o operador solução $G^\dagger : \mathcal{A} \rightarrow \mathcal{U}$, remodelamos o problema como o mapeamento $(\alpha, \beta) \mapsto (\eta, u)$. Um operador neural o aproxima por

$$G_\theta = Q \circ \sigma(W_L + \mathcal{K}_L) \circ \cdots \circ \sigma(W_1 + \mathcal{K}_1) \circ P,$$

em que cada camada atualiza os canais por

$$v_{\ell+1}(x) = \sigma\left((W_\ell v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\right).$$

- ▶ $P : \mathbb{R}^{d_a} \rightarrow \mathbb{R}^{d_c}$: operador de elevação, leva a entrada ponto a ponto a $d_c \gg d_a$ canais;
- ▶ W_ℓ : mapa linear local; σ : ativação aplicada ponto a ponto;
- ▶ $\mathcal{K}_\ell$: operador integral não local, com kernel aprendido $\kappa_\ell : D \times D \rightarrow \mathbb{R}^{d_c \times d_c}$, que faz cada ponto da saída ler a função inteira:

$$(\mathcal{K}_\ell v)(x) = \int_D \kappa_\ell(x, y) v(y) dy;$$

- ▶ $Q : \mathbb{R}^{d_c} \rightarrow \mathbb{R}^{d_u}$: projeção de volta à dimensão da saída;

Formulação Matemática de Operadores Neurais

Sejam $\mathcal{A}$ e $\mathcal{U}$ espaços de Banach de funções sobre D . O alvo é o operador solução $G^\dagger : \mathcal{A} \rightarrow \mathcal{U}$, remodelamos o problema como o mapeamento $(\alpha, \beta) \mapsto (\eta, u)$. Um operador neural o aproxima por

$$G_\theta = Q \circ \sigma(W_L + \mathcal{K}_L) \circ \cdots \circ \sigma(W_1 + \mathcal{K}_1) \circ P,$$

em que cada camada atualiza os canais por

$$v_{\ell+1}(x) = \sigma\left((W_\ell v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\right).$$

- ▶ $P : \mathbb{R}^{d_a} \rightarrow \mathbb{R}^{d_c}$: operador de elevação, leva a entrada ponto a ponto a $d_c \gg d_a$ canais;
- ▶ W_ℓ : mapa linear local; σ : ativação aplicada ponto a ponto;
- ▶ $\mathcal{K}_\ell$: operador integral não local, com kernel aprendido $\kappa_\ell : D \times D \rightarrow \mathbb{R}^{d_c \times d_c}$, que faz cada ponto da saída ler a função inteira:

$$(\mathcal{K}_\ell v)(x) = \int_D \kappa_\ell(x, y) v(y) dy;$$

- ▶ $Q : \mathbb{R}^{d_c} \rightarrow \mathbb{R}^{d_u}$: projeção de volta à dimensão da saída;
- ▶ Cada parametrização de κ_ℓ dá uma arquitetura.

O FNO (Li et al. 2021) toma o kernel invariante por translação, $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. O operador $\mathcal{K}_\ell$ é uma convolução de κ_ℓ com v_ℓ , então

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y) v_\ell(y) dy.$$

O FNO (Li et al. 2021) toma o kernel invariante por translação, $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. O operador $\mathcal{K}_\ell$ é uma convolução de κ_ℓ com v_ℓ , então

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y) v_\ell(y) dy.$$

Isto é,

$$(\mathcal{K}_\ell v_\ell)(x) = \mathcal{F}^{-1} \left(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell) \right)(x).$$

O FNO (Li et al. 2021) toma o kernel invariante por translação, $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. O operador $\mathcal{K}_\ell$ é uma convolução de κ_ℓ com v_ℓ , então

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y) v_\ell(y) dy.$$

Isto é,

$$(\mathcal{K}_\ell v_\ell)(x) = \mathcal{F}^{-1} \left(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell) \right)(x).$$

- Coeficientes de Fourier do kernel são matrizes treináveis $R_{\ell,k}$ truncadas em $|k| \leq k_{\max}$;

O FNO (Li et al. 2021) toma o kernel invariante por translação, $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. O operador $\mathcal{K}_\ell$ é uma convolução de κ_ℓ com v_ℓ , então

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y) v_\ell(y) dy.$$

Isto é,

$$(\mathcal{K}_\ell v_\ell)(x) = \mathcal{F}^{-1} \left(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell) \right) (x).$$

- ▶ Coeficientes de Fourier do kernel são matrizes treináveis $R_{\ell,k}$ truncadas em $|k| \leq k_{\max}$;
- ▶ Invariância à discretização, mas mantendo padrão para FFTs de 2^N pontos.

O FNO (Li et al. 2021) toma o kernel invariante por translação, $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. O operador $\mathcal{K}_\ell$ é uma convolução de κ_ℓ com v_ℓ , então

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y) v_\ell(y) dy.$$

Isto é,

$$(\mathcal{K}_\ell v_\ell)(x) = \mathcal{F}^{-1} \left(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell) \right)(x).$$

- ▶ Coeficientes de Fourier do kernel são matrizes treináveis $R_{\ell,k}$ truncadas em $|k| \leq k_{\max}$;
- ▶ Invariância à discretização, mas mantendo padrão para FFTs de 2^N pontos.

Treino puramente supervisionado, com $(\eta_\theta^i, u_\theta^i) = G_\theta(\alpha_i, \beta_i)$:

$$\mathcal{L}_{\text{FNO}}(\theta) = \frac{1}{M} \sum_{i=1}^M \frac{1}{N_x N_t} \sum_{j=0}^{N_x-1} \sum_{n=0}^{N_t} \left[\left(\eta_\theta^i(x_j, t_n) - \eta_i(x_j, t_n) \right)^2 + \left(u_\theta^i(x_j, t_n) - u_i(x_j, t_n) \right)^2 \right].$$

O PINO (Li et al. 2023a) mantém o esqueleto do FNO e soma resíduo e condição inicial ao termo de dados:

$$\mathcal{L}_{\text{PINO}}(\theta) = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}} \mathcal{L}_{\text{data}}(\theta).$$

- Para o sistema de Boussinesq, a saída são duas matrizes $N_x \times N_t$ da aproximação de η e u em um único forward pass para um dado $\alpha = \beta$ qualquer (é input também);

O PINO (Li et al. 2023a) mantém o esqueleto do FNO e soma resíduo e condição inicial ao termo de dados:

$$\mathcal{L}_{\text{PINO}}(\theta) = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}} \mathcal{L}_{\text{data}}(\theta).$$

- ▶ Para o sistema de Boussinesq, a saída são duas matrizes $N_x \times N_t$ da aproximação de η e u em um único forward pass para um dado $\alpha = \beta$ qualquer (é input também);
- ▶ Avaliar o resíduo da EDP é diferenciar numericamente essa matriz, não a rede;

O PINO (Li et al. 2023a) mantém o esqueleto do FNO e soma resíduo e condição inicial ao termo de dados:

$$\mathcal{L}_{\text{PINO}}(\theta) = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}} \mathcal{L}_{\text{data}}(\theta).$$

- ▶ Para o sistema de Boussinesq, a saída são duas matrizes $N_x \times N_t$ da aproximação de η e u em um único forward pass para um dado $\alpha = \beta$ qualquer (é input também);
- ▶ Avaliar o resíduo da EDP é diferenciar numericamente essa matriz, não a rede;
- ▶ Derivadas espaciais pseudoespectrais e temporais por diferenças finitas (Euler implícito, seguindo implementação original de Li et al. (2023a)) (Li et al. 2023b);

O PINO (Li et al. 2023a) mantém o esqueleto do FNO e soma resíduo e condição inicial ao termo de dados:

$$\mathcal{L}_{\text{PINO}}(\theta) = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}} \mathcal{L}_{\text{data}}(\theta).$$

- ▶ Para o sistema de Boussinesq, a saída são duas matrizes $N_x \times N_t$ da aproximação de η e u em um único forward pass para um dado $\alpha = \beta$ qualquer (é input também);
- ▶ Avaliar o resíduo da EDP é diferenciar numericamente essa matriz, não a rede;
- ▶ Derivadas espaciais pseudoespectrais e temporais por diferenças finitas (Euler implícito, seguindo implementação original de Li et al. (2023a)) (Li et al. 2023b);
- ▶ Permite aproveitar conhecimento das equações.

Viés espectral: estudo via fluxo do gradiente

A descida do gradiente com taxa de aprendizado $\mu > 0$ atualiza os parâmetros por

$$\theta^{(k+1)} = \theta^{(k)} - \mu \nabla_{\theta} \mathcal{L}(\theta^{(k)}) \iff \frac{\theta^{(k+1)} - \theta^{(k)}}{\mu} = -\nabla_{\theta} \mathcal{L}(\theta^{(k)}).$$

Viés espectral: estudo via fluxo do gradiente

A descida do gradiente com taxa de aprendizado $\mu > 0$ atualiza os parâmetros por

$$\theta^{(k+1)} = \theta^{(k)} - \mu \nabla_{\theta} \mathcal{L}(\theta^{(k)}) \iff \frac{\theta^{(k+1)} - \theta^{(k)}}{\mu} = -\nabla_{\theta} \mathcal{L}(\theta^{(k)}).$$

Lendo os iterados como amostras de uma curva $\theta(\tau)$ nos instantes $\tau^{(k)} = k\mu$, o lado esquerdo vira um quociente de diferenças progressivas:

$$\frac{\theta(\tau^{(k)} + \mu) - \theta(\tau^{(k)})}{\mu} = -\nabla_{\theta} \mathcal{L}(\theta(\tau^{(k)})).$$

Viés espectral: estudo via fluxo do gradiente

A descida do gradiente com taxa de aprendizado $\mu > 0$ atualiza os parâmetros por

$$\theta^{(k+1)} = \theta^{(k)} - \mu \nabla_{\theta} \mathcal{L}(\theta^{(k)}) \iff \frac{\theta^{(k+1)} - \theta^{(k)}}{\mu} = -\nabla_{\theta} \mathcal{L}(\theta^{(k)}).$$

Lendo os iterados como amostras de uma curva $\theta(\tau)$ nos instantes $\tau^{(k)} = k\mu$, o lado esquerdo vira um quociente de diferenças progressivas:

$$\frac{\theta(\tau^{(k)} + \mu) - \theta(\tau^{(k)})}{\mu} = -\nabla_{\theta} \mathcal{L}(\theta(\tau^{(k)})).$$

Com $\mu \rightarrow 0$, chegamos à EDO do fluxo do gradiente:

$$\frac{d\theta}{d\tau} = -\nabla_{\theta} \mathcal{L}(\theta), \quad \theta(0) = \theta^{(0)}.$$

Viés espectral: estudo via fluxo do gradiente

A descida do gradiente com taxa de aprendizado $\mu > 0$ atualiza os parâmetros por

$$\theta^{(k+1)} = \theta^{(k)} - \mu \nabla_{\theta} \mathcal{L}(\theta^{(k)}) \iff \frac{\theta^{(k+1)} - \theta^{(k)}}{\mu} = -\nabla_{\theta} \mathcal{L}(\theta^{(k)}).$$

Lendo os iterados como amostras de uma curva $\theta(\tau)$ nos instantes $\tau^{(k)} = k\mu$, o lado esquerdo vira um quociente de diferenças progressivas:

$$\frac{\theta(\tau^{(k)} + \mu) - \theta(\tau^{(k)})}{\mu} = -\nabla_{\theta} \mathcal{L}(\theta(\tau^{(k)})).$$

Com $\mu \rightarrow 0$, chegamos à EDO do fluxo do gradiente:

$$\frac{d\theta}{d\tau} = -\nabla_{\theta} \mathcal{L}(\theta), \quad \theta(0) = \theta^{(0)}.$$

A descida do gradiente é exatamente o método de Euler explícito aplicado a essa EDO, com passo μ (Süli e Mayers 2003; Burden e Faires 2011).

Com $e(\tau) = (u_\theta(x_i) - u_i)_{i=1}^N$ o vetor de erros pontuais e $J(\theta) = \partial e / \partial \theta$ seu Jacobiano, após algumas manipulações do fluxo do gradiente, considerando a regra da cadeia², chegamos a

$$\frac{de}{d\tau} = -J(\theta) J(\theta)^T e(\tau) \quad \implies \quad \frac{de}{d\tau} = -K(\theta) e(\tau), \quad \text{onde} \quad K_{ij}(\theta) = \nabla_\theta e_i^T \nabla_\theta e_j.$$

²Ver Seção 2.3.3 do texto do TCC.

Com $e(\tau) = (u_\theta(x_i) - u_i)_{i=1}^N$ o vetor de erros pontuais e $J(\theta) = \partial e / \partial \theta$ seu Jacobiano, após algumas manipulações do fluxo do gradiente, considerando a regra da cadeia², chegamos a

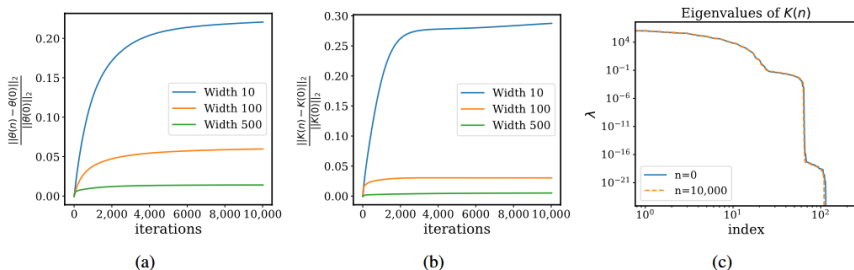
$$\frac{de}{d\tau} = -J(\theta) J(\theta)^T e(\tau) \implies \frac{de}{d\tau} = -K(\theta) e(\tau), \quad \text{onde} \quad K_{ij}(\theta) = \nabla_\theta e_i^T \nabla_\theta e_j.$$

$K(\theta)$ é o NTK empírico (Jacot et al. 2018): a matriz de Gram do produto interno dos gradientes.

²Ver Seção 2.3.3 do texto do TCC.

Viés espectral: NTK quase constante

Em redes largas, cada parâmetro se move pouco durante o treino, e o Jacobiano quase não muda: $K(\theta(\tau)) \approx K_0$ (Jacot et al. 2018; Chizat et al. 2019).



Variação relativa (a) dos parâmetros θ e (b) do NTK K de uma PINN para Poisson 1D; (c) autovalores do NTK na inicialização e ao fim do treino (Wang et al. 2022).

Assim,

$$\frac{de}{d\tau} = -K(\theta) e(\theta) \approx -K(\theta(\tau_0)) e(\tau),$$

obtendo uma EDO linear com coeficientes constantes para evolução do vetor de erros.

Com o kernel parado na inicialização, o erro segue $\frac{de}{d\tau} = -K_0 e(\tau)$. Como K_0 é simétrica e semidefinida positiva (Strang 2006; Golub e Van Loan 2013),

$$K_0 = V\Lambda V^T, \quad V^T V = I, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_N), \quad \lambda_1 \geq \dots \geq \lambda_N \geq 0.$$

Com o kernel parado na inicialização, o erro segue $\frac{de}{d\tau} = -K_0 e(\tau)$. Como K_0 é simétrica e semidefinida positiva (Strang 2006; Golub e Van Loan 2013),

$$K_0 = V\Lambda V^T, \quad V^T V = I, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_N), \quad \lambda_1 \geq \dots \geq \lambda_N \geq 0.$$

Mudando de base, $c(\tau) = V^T e(\tau)$ e $e(\tau) = Vc(\tau)$:

$$\frac{dc}{d\tau} = -\Lambda c(\tau).$$

Com o kernel parado na inicialização, o erro segue $\frac{de}{d\tau} = -K_0 e(\tau)$. Como K_0 é simétrica e semidefinida positiva (Strang 2006; Golub e Van Loan 2013),

$$K_0 = V \Lambda V^T, \quad V^T V = I, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_N), \quad \lambda_1 \geq \dots \geq \lambda_N \geq 0.$$

Mudando de base, $c(\tau) = V^T e(\tau)$ e $e(\tau) = V c(\tau)$:

$$\frac{dc}{d\tau} = -\Lambda c(\tau).$$

Como Λ é diagonal, o sistema se desacopla em N EDOs escalares:

$$\frac{dc_k}{d\tau} = -\lambda_k c_k(\tau) \implies c_k(\tau) = c_k(0) e^{-\lambda_k \tau}, \quad k = 1, \dots, N.$$

Voltando à base original, obtemos a solução completa da EDO $\frac{de}{d\tau} = -K_0 e(\tau)$:

$$e(\tau) = \sum_{k=1}^N (v_k^T e_0) e^{-\lambda_k \tau} v_k.$$

Voltando à base original, obtemos a solução completa da EDO $\frac{de}{d\tau} = -K_0 e(\tau)$:

$$e(\tau) = \sum_{k=1}^N (v_k^T e_0) e^{-\lambda_k \tau} v_k.$$

Autovetores com λ_k grande somem rápido; os de λ_k pequeno persistem por todo o treino (Cao et al. 2021).

Se os dados estão numa grade regular e a função alvo é periódica, o erro admite a DFT. Com a matriz de Fourier unitária F ($F^* F = I$), $\hat{e}(\tau) = F e(\tau)$ e

$$\frac{d\hat{e}}{d\tau} = F \frac{de}{d\tau} = -F K_0 F^{-1} \hat{e}(\tau), \quad F K_0 F^{-1} = (FV) \Lambda (FV)^{-1}.$$

Se os dados estão numa grade regular e a função alvo é periódica, o erro admite a DFT. Com a matriz de Fourier unitária F ($F^* F = I$), $\hat{e}(\tau) = F e(\tau)$ e

$$\frac{d\hat{e}}{d\tau} = F \frac{de}{d\tau} = -F K_0 F^{-1} \hat{e}(\tau), \quad F K_0 F^{-1} = (FV) \Lambda (FV)^{-1}.$$

As colunas Fv_k diagonalizam a dinâmica. Com $\tilde{c}(\tau) = (FV)^{-1} \hat{e}(\tau)$, os mesmos passos dão $\tilde{c}_k(\tau) = \tilde{c}_k(0) e^{-\lambda_k \tau}$ e

$$\hat{e}(\tau) = \sum_{k=1}^N [(Fv_k)^* \hat{e}_0] e^{-\lambda_k \tau} (Fv_k).$$

Se os dados estão numa grade regular e a função alvo é periódica, o erro admite a DFT. Com a matriz de Fourier unitária F ($F^*F = I$), $\hat{e}(\tau) = F e(\tau)$ e

$$\frac{d\hat{e}}{d\tau} = F \frac{de}{d\tau} = -FK_0F^{-1} \hat{e}(\tau), \quad FK_0F^{-1} = (FV) \Lambda (FV)^{-1}.$$

As colunas Fv_k diagonalizam a dinâmica. Com $\tilde{c}(\tau) = (FV)^{-1}\hat{e}(\tau)$, os mesmos passos dão $\tilde{c}_k(\tau) = \tilde{c}_k(0) e^{-\lambda_k \tau}$ e

$$\hat{e}(\tau) = \sum_{k=1}^N [(Fv_k)^* \hat{e}_0] e^{-\lambda_k \tau} (Fv_k).$$

Os autovetores de maior λ_k são suaves e concentrados nas baixas frequências (Rahaman et al. 2019; Xu et al. 2020): o conteúdo de baixa frequência do erro decai primeiro.

Se quisermos garantir que o erro na k -ésima componente seja menor que $\varepsilon > 0$, podemos isolar:

$$|v_k^T e(0)| e^{-\lambda_k \tau} < \varepsilon \quad \implies \quad \tau > \frac{1}{\lambda_k} \ln \frac{|v_k^T e(0)|}{\varepsilon}.$$

Se quisermos garantir que o erro na k -ésima componente seja menor que $\varepsilon > 0$, podemos isolar:

$$|v_k^T e(0)| e^{-\lambda_k \tau} < \varepsilon \implies \tau > \frac{1}{\lambda_k} \ln \frac{|v_k^T e(0)|}{\varepsilon}.$$

Mas esta estimativa é em τ ; fazendo a mesma análise para a versão discretizada, chegamos em:

$$|v_k^T e(0)| |1 - \mu \lambda_k|^n < \varepsilon \implies n > \frac{1}{-\ln |1 - \mu \lambda_k|} \ln \frac{|v_k^T e(0)|}{\varepsilon}.$$

Se quisermos garantir que o erro na k -ésima componente seja menor que $\varepsilon > 0$, podemos isolar:

$$|v_k^T e(0)| e^{-\lambda_k \tau} < \varepsilon \implies \tau > \frac{1}{\lambda_k} \ln \frac{|v_k^T e(0)|}{\varepsilon}.$$

Mas esta estimativa é em τ ; fazendo a mesma análise para a versão discretizada, chegamos em:

$$|v_k^T e(0)| |1 - \mu \lambda_k|^n < \varepsilon \implies n > \frac{1}{-\ln |1 - \mu \lambda_k|} \ln \frac{|v_k^T e(0)|}{\varepsilon}.$$

Aproveitando, como o método do gradiente aqui nada mais é do que o método de Euler explícito, a taxa de aprendizado (tamanho do passo) segue as mesmas estimativas que garantem a estabilidade do método, ou seja, $\mu < 2/\lambda_{\max}$ (Süli e Mayers 2003; LeVeque 2007); tomamos assim $\mu = 0,4/\lambda_{\max}$.

Metodologia

Todo o software por trás destes experimentos foi escrito para este trabalho.

Item	Escolha
Linguagem e biblioteca	Python 3.11, PyTorch 2.12.0 (Paszke et al. 2019)
Aceleração	CUDA 13.0, uma única GPU NVIDIA GeForce GTX 1660 SUPER
Solver de referência	pseudospectral com Runge–Kutta 4, código próprio
Arquiteturas	FNO (Li et al. 2021), PINO (Li et al. 2023a), PINN (Raissi et al. 2019)
Reprodutibilidade	semente global 37, cuDNN determinístico, estágios em ordem fixa

- ▶ Sem pesos pré-treinados, sem biblioteca de terceiros e sem conjunto de dados externo;

Todo o software por trás destes experimentos foi escrito para este trabalho.

Item	Escolha
Linguagem e biblioteca	Python 3.11, PyTorch 2.12.0 (Paszke et al. 2019)
Aceleração	CUDA 13.0, uma única GPU NVIDIA GeForce GTX 1660 SUPER
Solver de referência	pseudospectral com Runge–Kutta 4, código próprio
Arquiteturas	FNO (Li et al. 2021), PINO (Li et al. 2023a), PINN (Raissi et al. 2019)
Reprodutibilidade	semente global 37, cuDNN determinístico, estágios em ordem fixa

- ▶ Sem pesos pré-treinados, sem biblioteca de terceiros e sem conjunto de dados externo;
- ▶ Escrever tudo do zero permitiu organizar o processo de otimização para medir o viés espectral;

Todo o software por trás destes experimentos foi escrito para este trabalho.

Item	Escolha
Linguagem e biblioteca	Python 3.11, PyTorch 2.12.0 (Paszke et al. 2019)
Aceleração	CUDA 13.0, uma única GPU NVIDIA GeForce GTX 1660 SUPER
Solver de referência	pseudospectral com Runge–Kutta 4, código próprio
Arquiteturas	FNO (Li et al. 2021), PINO (Li et al. 2023a), PINN (Raissi et al. 2019)
Reprodutibilidade	semente global 37, cuDNN determinístico, estágios em ordem fixa

- ▶ Sem pesos pré-treinados, sem biblioteca de terceiros e sem conjunto de dados externo;
- ▶ Escrever tudo do zero permitiu organizar o processo de otimização para medir o viés espectral;
- ▶ Implementação em <https://github.com/samuelkutz/TCC>.

Os métodos são treinados e avaliados no conjunto

$$\mathcal{S} = \left\{ ((\alpha_i, \beta_i), (\eta_i, u_i)) \right\}_{i=1}^M.$$

- ▶ (α_i, β_i) : parâmetros de não linearidade e de dispersão da i -ésima amostra;

Os métodos são treinados e avaliados no conjunto

$$\mathcal{S} = \{((\alpha_i, \beta_i), (\eta_i, u_i))\}_{i=1}^M.$$

- ▶ (α_i, β_i) : parâmetros de não linearidade e de dispersão da i -ésima amostra;
- ▶ (η_i, u_i) : solução espaço-temporal obtida pelo solver pseudoespectral com esses parâmetros;

Os métodos são treinados e avaliados no conjunto

$$\mathcal{S} = \left\{ ((\alpha_i, \beta_i), (\eta_i, u_i)) \right\}_{i=1}^M.$$

- ▶ (α_i, β_i) : parâmetros de não linearidade e de dispersão da i -ésima amostra;
- ▶ (η_i, u_i) : solução espaço-temporal obtida pelo solver pseudoespectral com esses parâmetros;
- ▶ A condição inicial é a mesma em todas os dados;

Os métodos são treinados e avaliados no conjunto

$$\mathcal{S} = \left\{ \left((\alpha_i, \beta_i), (\eta_i, u_i) \right) \right\}_{i=1}^M.$$

- ▶ (α_i, β_i) : parâmetros de não linearidade e de dispersão da i -ésima amostra;
- ▶ (η_i, u_i) : solução espaço-temporal obtida pelo solver pseudoespectral com esses parâmetros;
- ▶ A condição inicial é a mesma em todas os dados;
- ▶ As soluções de referência são os alvos supervisionados dos métodos orientados a dados.

Domínio $x \in [-60, 60]$, $t \in [0, 30]$, condição inicial $\eta(x, 0) = \text{sech}^2(x)$, $u(x, 0) = 0$. A largura é deliberada: os pulsos não alcançam a fronteira periódica no horizonte considerado.

- Física do PINO avaliado em $N_x = 128$ e $N_t = 128$ pontos, $\Delta x = 0,9375$ e $\Delta t \approx 0,2362$, em comparação ao domínio de tamanho $2L = 120$;

Domínio $x \in [-60, 60]$, $t \in [0, 30]$, condição inicial $\eta(x, 0) = \text{sech}^2(x)$, $u(x, 0) = 0$. A largura é deliberada: os pulsos não alcançam a fronteira periódica no horizonte considerado.

- ▶ Física do PINO avaliado em $N_x = 128$ e $N_t = 128$ pontos, $\Delta x = 0,9375$ e $\Delta t \approx 0,2362$, em comparação ao domínio de tamanho $2L = 120$;
- ▶ Parâmetros variando uniformemente em $\alpha = \beta \in [0, 1; 4, 0]$, com $M = 20$ amostras;

Domínio $x \in [-60, 60]$, $t \in [0, 30]$, condição inicial $\eta(x, 0) = \text{sech}^2(x)$, $u(x, 0) = 0$. A largura é deliberada: os pulsos não alcançam a fronteira periódica no horizonte considerado.

- ▶ Física do PINO avaliado em $N_x = 128$ e $N_t = 128$ pontos, $\Delta x = 0,9375$ e $\Delta t \approx 0,2362$, em comparação ao domínio de tamanho $2L = 120$;
- ▶ Parâmetros variando uniformemente em $\alpha = \beta \in [0, 1; 4, 0]$, com $M = 20$ amostras;
- ▶ Para NOs, há quatro canais de input $(\eta_i(x, 0), u_i(x, 0), \alpha_i, \beta_i)$ e dois de saída:

$$X \in \mathbb{R}^{M \times 4 \times N_x \times N_t}, \quad Y \in \mathbb{R}^{M \times 2 \times N_x \times N_t};$$

Domínio $x \in [-60, 60]$, $t \in [0, 30]$, condição inicial $\eta(x, 0) = \text{sech}^2(x)$, $u(x, 0) = 0$. A largura é deliberada: os pulsos não alcançam a fronteira periódica no horizonte considerado.

- ▶ Física do PINO avaliado em $N_x = 128$ e $N_t = 128$ pontos, $\Delta x = 0,9375$ e $\Delta t \approx 0,2362$, em comparação ao domínio de tamanho $2L = 120$;
- ▶ Parâmetros variando uniformemente em $\alpha = \beta \in [0, 1; 4, 0]$, com $M = 20$ amostras;
- ▶ Para NOs, há quatro canais de input ($\eta_i(x, 0)$, $u_i(x, 0)$, α_i , β_i) e dois de saída:

$$X \in \mathbb{R}^{M \times 4 \times N_x \times N_t}, \quad Y \in \mathbb{R}^{M \times 2 \times N_x \times N_t};$$

- ▶ Entradas e saídas normalizadas para $[-1, 1]$ e desnormalizadas no final.

Cada método é medido contra uma referência pseudoespectral recalculada, em $\alpha = \beta \in \{0,1; 3,21; 4,2\}$ e $N_x \in \{128, 256, 512\}$.

- ▶ Erro relativo resolvido no tempo:

$$E(t_n) = \frac{\|\eta_{\text{pred}}(\cdot, t_n) - \eta_{\text{true}}(\cdot, t_n)\|_2}{\|\eta_{\text{true}}(\cdot, t_n)\|_2};$$

Cada método é medido contra uma referência pseudoespectral recalculada, em $\alpha = \beta \in \{0,1; 3,21; 4,2\}$ e $N_x \in \{128, 256, 512\}$.

- ▶ Erro relativo resolvido no tempo:

$$E(t_n) = \frac{\|\eta_{\text{pred}}(\cdot, t_n) - \eta_{\text{true}}(\cdot, t_n)\|_2}{\|\eta_{\text{true}}(\cdot, t_n)\|_2};$$

- ▶ Erro espectral absoluto por modo ($1/N_x$ desfaz o escalonamento da DFT):

$$E_{\text{spec}}(k, t_n) = \frac{1}{N_x} \left| |\hat{\eta}_{\text{pred}}(k, t_n)| - |\hat{\eta}_{\text{true}}(k, t_n)| \right|;$$

Cada método é medido contra uma referência pseudoespectral recalculada, em $\alpha = \beta \in \{0,1; 3,21; 4,2\}$ e $N_x \in \{128, 256, 512\}$.

- ▶ Erro relativo resolvido no tempo:

$$E(t_n) = \frac{\|\eta_{\text{pred}}(\cdot, t_n) - \eta_{\text{true}}(\cdot, t_n)\|_2}{\|\eta_{\text{true}}(\cdot, t_n)\|_2};$$

- ▶ Erro espectral absoluto por modo ($1/N_x$ desfaz o escalonamento da DFT):

$$E_{\text{spec}}(k, t_n) = \frac{1}{N_x} \left| |\hat{\eta}_{\text{pred}}(k, t_n)| - |\hat{\eta}_{\text{true}}(k, t_n)| \right|;$$

- ▶ Erro médio por banda B (baixa, média, alta), a cada 500 iterações:

$$E_{\text{band}}(B) = \frac{1}{|B|} \sum_{k \in B} \frac{1}{N_t} \sum_{n=1}^{N_t} E_{\text{spec}}(k, t_n).$$

Quatro arquiteturas dão seis métodos treinados:

Família	Regime	Método	Parâmetros	Alvo do treino	Pesos pde/ic/data
Redes neurais	dados puros	MLP	198.401	$\alpha = \beta = 3,21$	- / - / 1
Redes neurais	dados e física	PINN	198.658	$\alpha = \beta = 3,21$	1 / 1 / 1
Redes neurais	física pura	PINN	198.658	$\alpha = \beta = 3,21$	1 / 1 / 0
Operadores	dados puros	FNO	2.106.146	as 20 amostras	- / - / 1
Operadores	dados e física	PINO	2.106.146	as 20 amostras	100 / 1 / 1
Operadores	física pura	PINO	2.106.146	as 20 amostras	100 / 1 / 0

- ▶ 15.000 iterações de Adam com taxa de aprendizado 10^{-3} ;

Quatro arquiteturas dão seis métodos treinados:

Família	Regime	Método	Parâmetros	Alvo do treino	Pesos pde/ic/data
Redes neurais	dados puros	MLP	198.401	$\alpha = \beta = 3,21$	— / — / 1
Redes neurais	dados e física	PINN	198.658	$\alpha = \beta = 3,21$	1 / 1 / 1
Redes neurais	física pura	PINN	198.658	$\alpha = \beta = 3,21$	1 / 1 / 0
Operadores	dados puros	FNO	2.106.146	as 20 amostras	— / — / 1
Operadores	dados e física	PINO	2.106.146	as 20 amostras	100 / 1 / 1
Operadores	física pura	PINO	2.106.146	as 20 amostras	100 / 1 / 0

- ▶ 15.000 iterações de Adam com taxa de aprendizado 10^{-3} ;
- ▶ Os operadores leem as 20 amostras de uma vez; MLP e PINNs fixados a $\alpha = \beta = 3,21$.

Treinamos uma rede $\mathbb{R} \rightarrow \mathbb{R}$ aproximando contra o perfil de elevação em tempo fixo, solução vinda do pseudoespectral.

$$x \mapsto \eta_{\theta}(x) \approx \eta(x, 15), \quad x \in [-60, 60], \quad \alpha = \beta = 3,21.$$

- ▶ Três larguras, $N_{\text{MLP}} \in \{8, 64, 512\}$ para 4 camadas ocultas;

Treinamos uma rede $\mathbb{R} \rightarrow \mathbb{R}$ aproximando contra o perfil de elevação em tempo fixo, solução vinda do pseudoespectral.

$$x \mapsto \eta_{\theta}(x) \approx \eta(x, 15), \quad x \in [-60, 60], \quad \alpha = \beta = 3,21.$$

- ▶ Três larguras, $N_{\text{MLP}} \in \{8, 64, 512\}$ para 4 camadas ocultas;
- ▶ Descida do gradiente pura, full batch, 500.000 iterações;

Experimento para averiguar viés espectral

Treinamos uma rede $\mathbb{R} \rightarrow \mathbb{R}$ aproximando contra o perfil de elevação em tempo fixo, solução vinda do pseudoespectral.

$$x \mapsto \eta_{\theta}(x) \approx \eta(x, 15), \quad x \in [-60, 60], \quad \alpha = \beta = 3,21.$$

- ▶ Três larguras, $N_{\text{MLP}} \in \{8, 64, 512\}$ para 4 camadas ocultas;
- ▶ Descida do gradiente pura, full batch, 500.000 iterações;
- ▶ Passo $\mu = 0,4/\lambda_{\max}$, com $\lambda_{\max}$ sendo o maior autovalor do kernel inicial;

Treinamos uma rede $\mathbb{R} \rightarrow \mathbb{R}$ aproximando contra o perfil de elevação em tempo fixo, solução vinda do pseudoespectral.

$$x \mapsto \eta_{\theta}(x) \approx \eta(x, 15), \quad x \in [-60, 60], \quad \alpha = \beta = 3,21.$$

- ▶ Três larguras, $N_{\text{MLP}} \in \{8, 64, 512\}$ para 4 camadas ocultas;
- ▶ Descida do gradiente pura, full batch, 500.000 iterações;
- ▶ Passo $\mu = 0,4/\lambda_{\max}$, com $\lambda_{\max}$ sendo o maior autovalor do kernel inicial;
- ▶ O kernel $K_0 = J_0 J_0^T$ fica com 128×128 e pode ser diagonalizado diretamente, por ser semidefinido positivo.

Parâmetro	Valor
<i>Domínio e condição inicial</i>	
Semicomprimento espacial L ($\Omega = [-L, L]$)	60
Tempo final T ($t \in [0, T]$)	30
Amplitude inicial A	1
Condição inicial	$\eta(x, 0) = \text{sech}^2(x), \quad u(x, 0) = 0$
<i>Solução de referência</i>	
Pontos da grade espacial N_x	128
Níveis de tempo (127 passos de RK4)	128
Espaçamento espacial $\Delta x = 2L/N_x$	0,9375
Espaçamento temporal $\Delta t = T/127$	$\approx 0,2362$
<i>Amostragem, avaliação e reprodutibilidade</i>	
Coefficientes acoplados	$\alpha = \beta$
Amostras de treino M / grade de amostragem	20 / linspace(0,1; 4,0; 20)
Valores de avaliação $\alpha = \beta$	{0,1; 3,21; 4,2}
Resoluções de avaliação N_x	{128, 256, 512}
Resolução do recorte espectral / semente global	256 / 37

Resultados

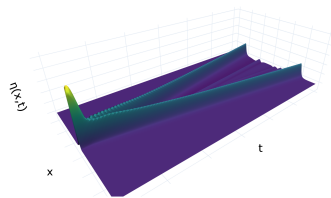
Primeiro, vamos ver o quanto cada método custa para treinar.

Método	Regime	Parâmetros	Tempo de treino (s)	Perda final
MLP	dados puros	198.401	151,9	$4,3 \times 10^{-5}$
PINN	dados e física	198.658	898,9	$7,1 \times 10^{-5}$
PINN	física pura	198.658	868,4	$4,6 \times 10^{-5}$
FNO	dados puros	2.106.146	1177,8	$4,8 \times 10^{-7}$
PINO	dados e física	2.106.146	5459,4	$5,2 \times 10^{-5}$
PINO	física pura	2.106.146	4441,2	$4,0 \times 10^{-5}$

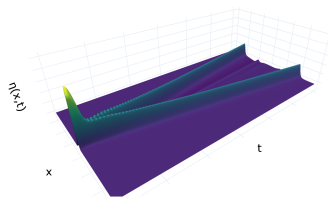
i Observação

Cada loss é uma escala diferente para um método diferente, não pode ser usado como comparativo.

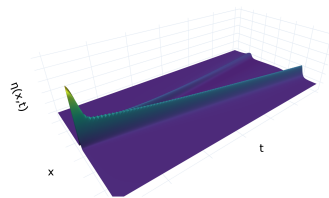
Treinados em resolução 128 para $\alpha = \beta = 3,21$, avaliados em 256.



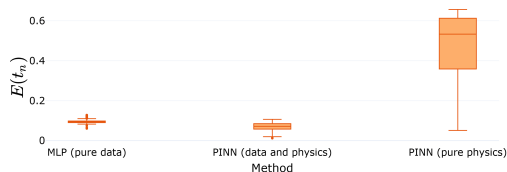
MLP (dados puros)



PINN (dados e física)

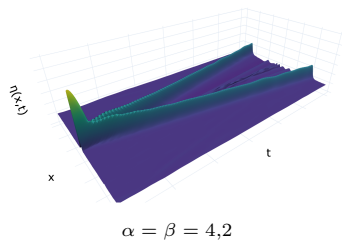
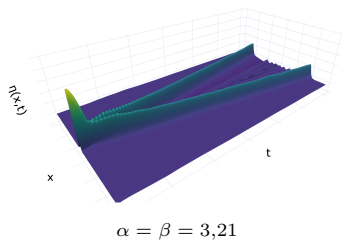
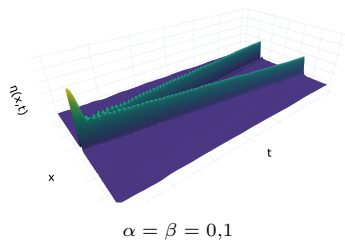


PINN (física pura)



FNO: variação dos parâmetros da EDP

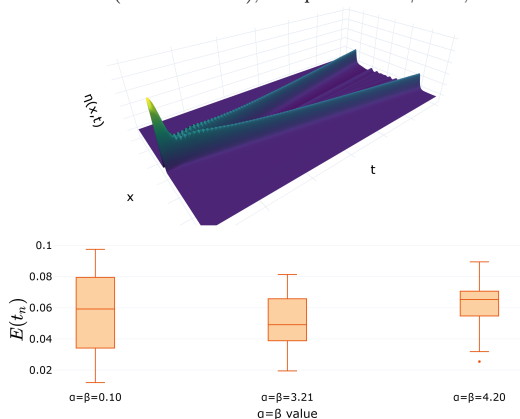
O FNO treinado só em dados, em três valores de $\alpha = \beta$ na resolução 256.



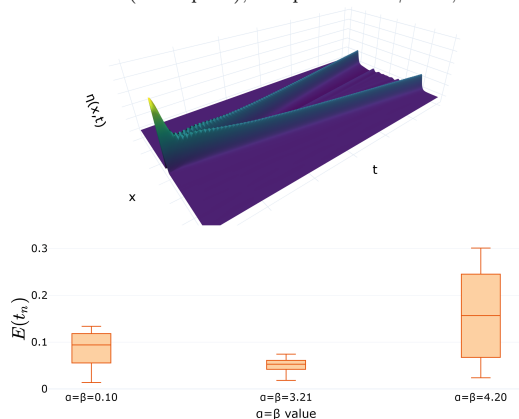
PINOs: variação dos parâmetros da EDP

Os dois regimes do PINO na mesma varredura e resolução.

PINO (dados e física), campo em $\alpha = \beta = 3,21$



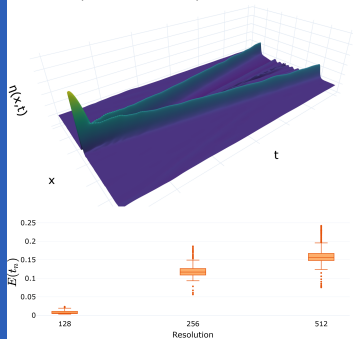
PINO (física pura), campo em $\alpha = \beta = 3,21$



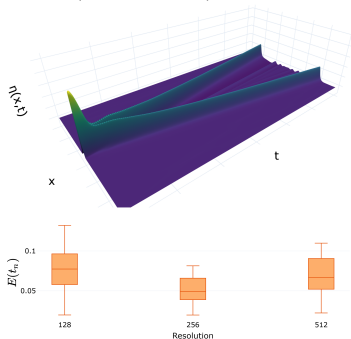
FNO e PINOs: variação de resolução

Consulta fora da grade de treino, sem retreino, em $\alpha = \beta = 3,21$.

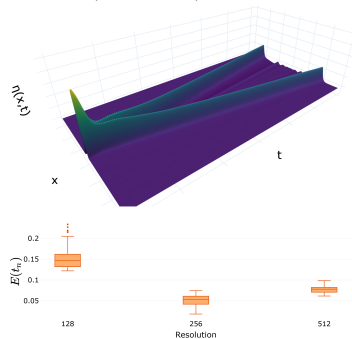
FNO (dados puros), resolução 512



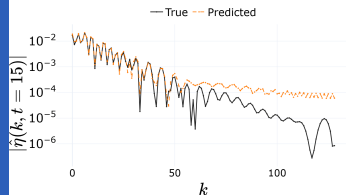
PINO (dados e física), resolução 512



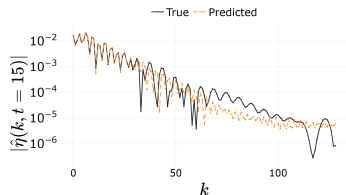
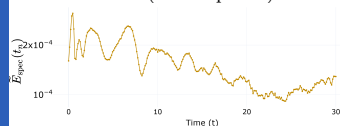
PINO (física pura), resolução 512



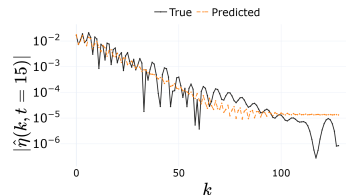
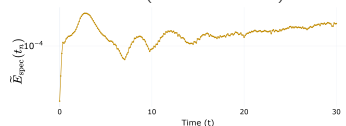
Comparação de espectros para $\alpha = \beta = 3,21$ e $t = 15$; abaixo, o erro absoluto médio sobre todos os modos ao longo do tempo.



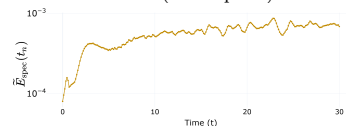
MLP (dados puros)



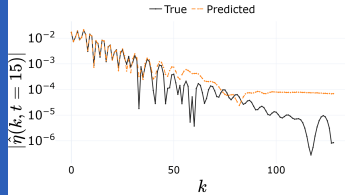
PINN (dados e física)



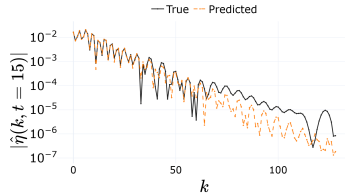
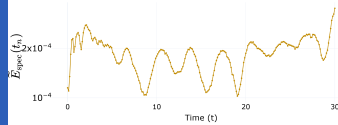
PINN (física pura)



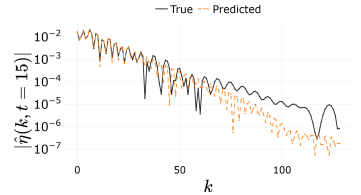
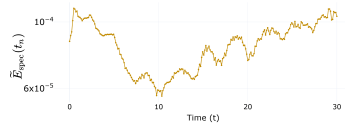
Mesmo parâmetro, mesmo instante e mesma leitura, agora para os operadores.



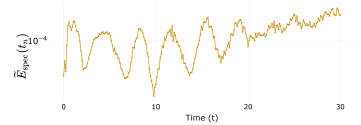
FNO (dados puros)



PINO (dados e física)

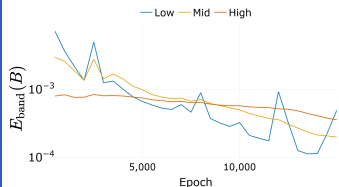


PINO (física pura)

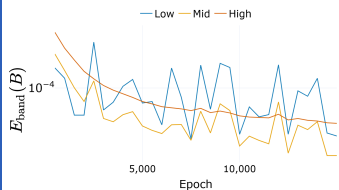


Viés espectral nos seis métodos

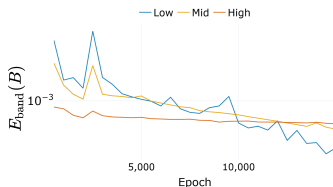
Erro absoluto médio por banda ao longo do treino, em $\alpha = \beta = 3,21$ e resolução 256.



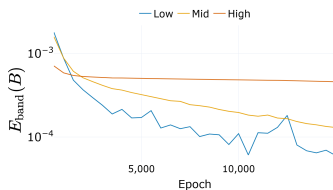
MLP (dados puros)



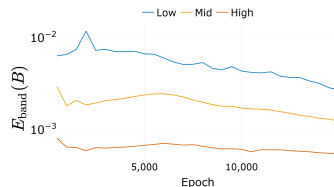
FNO (dados puros)



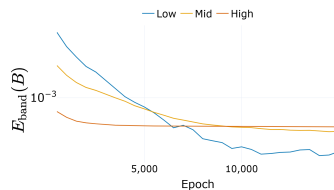
PINN (dados e física)



PINO (dados e física)



PINN (física pura)

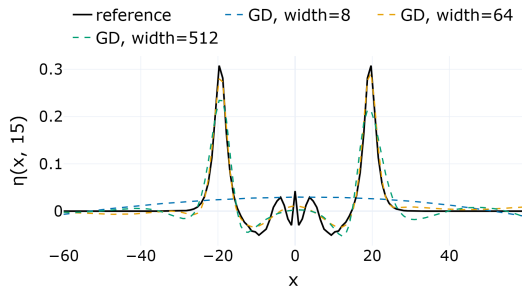


PINO (física pura)

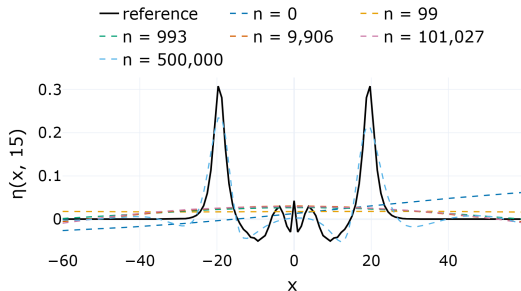
Bandas baixa (*Low*), média (*Mid*) e alta (*High*). A baixa cai rápido, já a banda alta permanece relativamente plana.

Viés espectral: aproximação por MLPs

As MLPs do experimento de viés espectral contra a solução de referência $\eta(x, 15)$ em $\alpha = \beta = 3,21$.



Ajuste final em cada largura

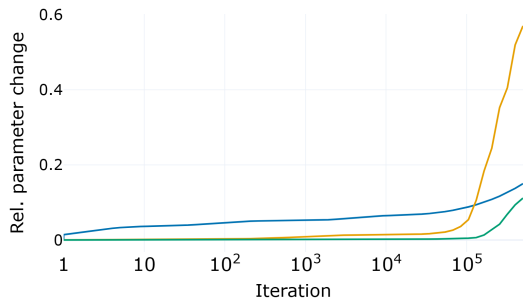


Rede mais larga ao longo do treino

- ▶ A largura 64 dá o melhor ajuste (por ter menos overfitting);
- ▶ O que interessa aqui é se as redes largas seguem o comportamento esperado de um kernel quase constante durante o treino.

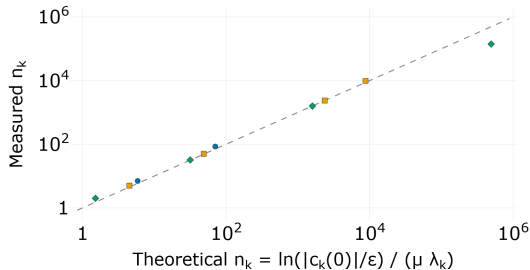
Viés espectral: deriva dos parâmetros e iterações

Deriva dos parâmetros e contagem de iterações ao longo dos treinos do experimento mínimo.



— width 8 — width 64 — width 512

Deriva dos parâmetros, $\|\theta - \theta_0\|/\|\theta_0\|$

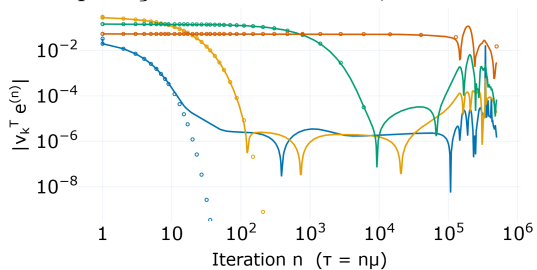


- - $y = x$ ● width 8 ■ width 64 ◆ width 512

Iterações: previsto contra medido

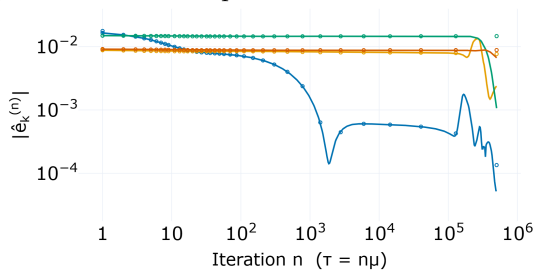
- ▶ Quanto mais larga a rede, menos os parâmetros e o kernel se movem.

As previsões do kernel inicial, medidas ao longo dos treinos do experimento mínimo.



— $k=1$ — $k=2$ — $k=3$ — $k=5$ — measured ○ prediction

Decaimento por autovetor, $|v_k^T e^{(n)}|$



— $k=1$ — $k=4$ — $k=9$ — $k=15$ — measured ○ prediction

Decaimento por modo de Fourier, $|\hat{e}_k^{(n)}|$

Conclusão

Em resumo, o ganhador é quando juntamos física com dados. **Redes neurais (MLP e PINN)**

- ▶ **MLP, dados puros.** Reproduz a onda; banda baixa depressa, alta devagar;

Operadores (FNO e PINO)

Em resumo, o ganhador é quando juntamos física com dados. **Redes neurais (MLP e PINN)**

- ▶ **MLP, dados puros.** Reproduz a onda; banda baixa depressa, alta devagar;
- ▶ **PINN, dados e física.** Menor erro relativo entre os três;

Operadores (FNO e PINO)

Em resumo, o ganhador é quando juntamos física com dados. **Redes neurais (MLP e PINN)**

- ▶ **MLP, dados puros.** Reproduz a onda; banda baixa depressa, alta devagar;
- ▶ **PINN, dados e física.** Menor erro relativo entre os três;
- ▶ **PINN, física pura.** Sem dados, não aprende detalhes da onda.

Operadores (FNO e PINO)

Em resumo, o ganhador é quando juntamos física com dados. **Redes neurais (MLP e PINN)**

- ▶ **MLP, dados puros.** Reproduz a onda; banda baixa depressa, alta devagar;
- ▶ **PINN, dados e física.** Menor erro relativo entre os três;
- ▶ **PINN, física pura.** Sem dados, não aprende detalhes da onda.

Operadores (FNO e PINO)

- ▶ **FNO.** Ajusta bem os dados, mas com viés espectral;

Em resumo, o ganhador é quando juntamos física com dados. **Redes neurais (MLP e PINN)**

- ▶ **MLP, dados puros.** Reproduz a onda; banda baixa depressa, alta devagar;
- ▶ **PINN, dados e física.** Menor erro relativo entre os três;
- ▶ **PINN, física pura.** Sem dados, não aprende detalhes da onda.

Operadores (FNO e PINO)

- ▶ **FNO.** Ajusta bem os dados, mas com viés espectral;
- ▶ **PINO, física pura.** Escapam as ondulações de alta frequência;

Em resumo, o ganhador é quando juntamos física com dados. **Redes neurais (MLP e PINN)**

- ▶ **MLP, dados puros.** Reproduz a onda; banda baixa depressa, alta devagar;
- ▶ **PINN, dados e física.** Menor erro relativo entre os três;
- ▶ **PINN, física pura.** Sem dados, não aprende detalhes da onda.

Operadores (FNO e PINO)

- ▶ **FNO.** Ajusta bem os dados, mas com viés espectral;
- ▶ **PINO, física pura.** Escapam as ondulações de alta frequência;
- ▶ **PINO, dados e física.** O caso mais consistente na varredura;

Em resumo, o ganhador é quando juntamos física com dados. **Redes neurais (MLP e PINN)**

- ▶ **MLP, dados puros.** Reproduz a onda; banda baixa depressa, alta devagar;
- ▶ **PINN, dados e física.** Menor erro relativo entre os três;
- ▶ **PINN, física pura.** Sem dados, não aprende detalhes da onda.

Operadores (FNO e PINO)

- ▶ **FNO.** Ajusta bem os dados, mas com viés espectral;
- ▶ **PINO, física pura.** Escapam as ondulações de alta frequência;
- ▶ **PINO, dados e física.** O caso mais consistente na varredura;
- ▶ **Invariância à discretização.** Fora da grade de treino, queda modesta.

Temos que levar em consideração que:

- ▶ Utilizamos uma única condição inicial;

Temos que levar em consideração que:

- ▶ Utilizamos uma única condição inicial;
- ▶ Consideramos $\alpha = \beta$;

Temos que levar em consideração que:

- ▶ Utilizamos uma única condição inicial;
- ▶ Consideramos $\alpha = \beta$;
- ▶ A solução de referência é numérica;

Temos que levar em consideração que:

- ▶ Utilizamos uma única condição inicial;
- ▶ Consideramos $\alpha = \beta$;
- ▶ A solução de referência é numérica;
- ▶ O estudo do viés espectral é bastante simplificado.

Temos que levar em consideração que:

- ▶ Utilizamos uma única condição inicial;
- ▶ Consideramos $\alpha = \beta$;
- ▶ A solução de referência é numérica;
- ▶ O estudo do viés espectral é bastante simplificado.

i Viés espectral

O estudo de viés espectral que fizemos é um cenário muito simplificado, não pode-se garantir que as estimativas encontradas se estendam para outras arquiteturas.

Vimos que o viés espectral é um comportamento real dessas arquiteturas, quem as implementa deve estar, ao menos, ciente disso.

- ▶ Nada neste trabalho favorece que ML substitui métodos clássicos de solução de EDPs;

i Fechamento

Conforme grande crescimento de ML, esperamos que este trabalho sirva como guia para estudantes que desejam conhecer a aplicação de SciML à EDPs.

Vimos que o viés espectral é um comportamento real dessas arquiteturas, quem as implementa deve estar, ao menos, ciente disso.

- ▶ Nada neste trabalho favorece que ML substitui métodos clássicos de solução de EDPs;
- ▶ Métodos clássicos vêm com a vantagem teórica de estimativas de erros e garantias de convergência;

i Fechamento

Conforme grande crescimento de ML, esperamos que este trabalho sirva como guia para estudantes que desejam conhecer a aplicação de SciML à EDPs.

Vimos que o viés espectral é um comportamento real dessas arquiteturas, quem as implementa deve estar, ao menos, ciente disso.

- ▶ Nada neste trabalho favorece que ML substitui métodos clássicos de solução de EDPs;
- ▶ Métodos clássicos vêm com a vantagem teórica de estimativas de erros e garantias de convergência;
- ▶ As comparações deste trabalho são amarradas apenas para o sistema de Boussinesq, não se pode generalizar para outras EDPs.

i Fechamento

Conforme grande crescimento de ML, esperamos que este trabalho sirva como guia para estudantes que desejam conhecer a aplicação de SciML à EDPs.

- Arora, S. et al. (2019). “Fine-Grained Analysis of Optimization and Generalization for Overparameterized Two-Layer Neural Networks”. Em: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. Vol. 97, pp. 322–332.
- Baker, N. et al. (2019). *Workshop Report on Basic Research Needs for Scientific Machine Learning: Core Technologies for Artificial Intelligence*. Rel. técn. Washington, DC: U.S. Department of Energy, Office of Science.
- Basri, R. et al. (2019). “The Convergence Rate of Neural Networks for Learned Functions of Different Frequencies”. Em: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. (Acesso em 02/09/2026).
- Bihlo, A. e R. O. Popovych (2022). “Physics-informed neural networks for the shallow-water equations on the sphere”. Em: *Journal of Computational Physics*. (Acesso em 02/09/2026).
- Bonev, B. et al. (2023). “Spherical Fourier Neural Operators: Learning Stable Dynamics on the Sphere”. Em: *International Conference on Machine Learning (ICML)*. (Acesso em 02/09/2026).
- Burden, R. L. e J. D. Faires (2011). *Numerical Analysis*. 9^a ed. Boston: Brooks/Cole.

- Cao, Y. et al. (2021). “Towards Understanding the Spectral Bias of Deep Learning”. Em: *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI)*. (Acesso em 02/09/2026).
- Chizat, L., E. Oyallon e F. Bach (2019). “On Lazy Training in Differentiable Programming”. Em: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. (Acesso em 02/09/2026).
- Dietrich, F. e W. Schilders (2025). “Scientific machine learning”. Em: *Mathematische Semesterberichte* 72, pp. 89–115.
- Golub, G. H. e C. F. Van Loan (2013). *Matrix Computations*. 4ª ed. Baltimore: Johns Hopkins University Press.
- Gupta, G. et al. (2021). “Multiwavelet-based Operator Learning for Differential Equations”. Em: *Advances in Neural Information Processing Systems (NeurIPS)*. (Acesso em 02/09/2026).
- Jacot, A., F. Gabriel e C. Hongler (2018). “Neural Tangent Kernel: Convergence and Generalization in Neural Networks”. Em: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 31. (Acesso em 02/09/2026).

- Jagtap, A. D. et al. (2022). “Deep Learning of Inverse Water Waves Problems Using Multi-Fidelity Data: Application to Serre–Green–Naghdi Equations”. Em: *arXiv preprint arXiv:2202.02899*. (Acesso em 02/09/2026).
- Karniadakis, G. E. et al. (2021). “Physics-informed machine learning”. Em: *Nature Reviews Physics* 3.6, pp. 422–440.
- Khodakarami, S. et al. (2026). “Spectral bias in physics-informed and operator learning: analysis and mitigation guidelines”. Em: *arXiv preprint*. (Acesso em 02/09/2026).
- Kingma, D. P. e J. Ba (2014). “Adam: A method for stochastic optimization”. Em: *arXiv preprint arXiv:1412.6980*.
- LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Philadelphia: Society for Industrial e Applied Mathematics.
- Li, Z. et al. (2023a). “Physics-Informed Neural Operator for Learning Partial Differential Equations”. Em: *ACM/IMS Transactions on Data Science*. (Acesso em 02/09/2026).
- (2023b). *PINO: Physics-Informed Neural Operator – Reference Implementation*. (Acesso em 02/09/2026).

- Li, Z. et al. (2021). “Fourier Neural Operator for parametric partial differential equations”. Em: *International Conference on Learning Representations (ICLR)*. (Acesso em 02/09/2026).
- Linnainmaa, A. (1970). “Taylor expansion of the accumulated rounding error”. Em: *BIT Numerical Mathematics* 10.1, pp. 47–55.
- Liu, H. et al. (2023). “Prediction of phase transition and time-varying dynamics of the (2+1)-dimensional Boussinesq equation by parameter-integrated PINNs with phase domain decomposition”. Em: *Physical Review E* 108.4, p. 045303.
- Lu, L. et al. (2021). “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators”. Em: *Nature Machine Intelligence* 3, pp. 218–229.
- Martins, L. G. (2023). “Estudo Numérico do Modelo Boussinesq com Topografia e Pressão Variável no Espaço e no Tempo”. Diss. de mestr. Curitiba: Universidade Federal do Paraná.
- Paszke, A. et al. (2019). “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. Em: *Annual Conference on Neural Information Processing Systems (NeurIPS)*. Vancouver, pp. 8024–8035. (Acesso em 02/09/2026).

- Pizzo, N. (19 de mai. de 2016). *Shallow water wave generation (quasi solitary wave with breaking)*. Vídeo do YouTube. URL: <https://www.youtube.com/watch?v=w-oDnVbV8mY>.
- Rahaman, N. et al. (2019). “On the spectral bias of neural networks”. Em: *International Conference on Machine Learning (ICML)*. PMLR. Long Beach, pp. 5301–5310.
- Raissi, M., P. Perdikaris e G. E. Karniadakis (2019). “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. Em: *Journal of Computational Physics* 378, pp. 686–707.
- Strang, G. (2006). *Linear Algebra and Its Applications*. 4^a ed. Belmont: Thomson Brooks/Cole.
- Su, J.-J. et al. (2024). “Explorations of certain nonlinear waves of the Boussinesq and Camassa–Holm equations using physics-informed neural networks”. Em: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 480.2284, p. 20230580.
- Süli, E. e D. F. Mayers (2003). *An Introduction to Numerical Analysis*. Cambridge: Cambridge University Press.
- Uchida, T. et al. (2025). “Ocean Emulation With Fourier Neural Operators: Double Gyre”. Em: *Journal of Advances in Modeling Earth Systems* 17.1.

- Wang, S., X. Yu e P. Perdikaris (2022). “When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective”. Em: *Journal of Computational Physics* 449, p. 110768.
- Xu, Z.-Q. J. et al. (2020). “Frequency Principle: Fourier Analysis Sheds Light on Deep Neural Networks”. Em: *Communications in Computational Physics* 28.5, pp. 1746–1767.
- Yu, J. et al. (2025). “Spherical multigrid neural operator for improving autoregressive global weather forecasting”. Em: *Scientific Reports* 15.1.

Obrigado pela atenção!

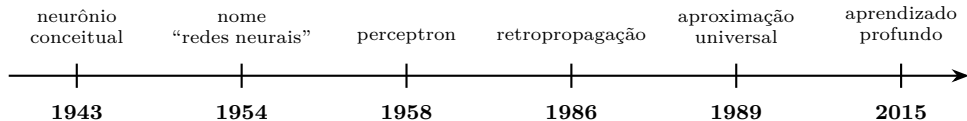


LABFluid
Laboratório de Dinâmica
dos Fluidos Computacional

Conteúdo

- 1 Linha do tempo das redes neurais
- 2 Decaimento do erro por autovetor do NTK
- 3 As três bandas de frequência
- 4 Normalização e onde cada método a aplica
- 5 Painéis de resolução do PINO
- 6 Método pseudoespectral

Toda arquitetura estudada aqui repousa sobre os mesmos blocos básicos.



- ▶ McCulloch e Pitts (1943) e Rosenblatt (1958): o neurônio matemático e o perceptron; o nome “redes neurais” já circula nos anos 1950 (Minsky 1954);
- ▶ Retropropagação (Rumelhart et al. 1986): múltiplas camadas por gradiente;
- ▶ Aproximação universal (Cybenko 1989; Hornik et al. 1989): um Multilayer Perceptron (MLP) largo o bastante aproxima qualquer função contínua em um compacto;
- ▶ “Profundo” (LeCun et al. 2015): muitas camadas ocultas, mais capacidade representacional, popularização de “Redes Neurais Profundas” (Deep Neural Networks).

Decaimento do erro por autovetor do NTK

Sob treinamento preguiçoso, $K(\theta(\tau)) \approx K_0$ e a dinâmica do erro vira $de/d\tau = -K_0 e$, com $e(0) = e_0$.

- ▶ K_0 é simétrica positiva semidefinida, logo o teorema espectral (Strang 2006; Golub e Van Loan 2013) dá $K_0 = V\Lambda V^T$, V ortogonal;
- ▶ A mudança $c(\tau) = V^T e(\tau)$ leva a $dc/d\tau = -\Lambda c$;
- ▶ Com Λ diagonal, o sistema desacopla em $dc_k/d\tau = -\lambda_k c_k$, de solução $c_k(\tau) = c_k(0)e^{-\lambda_k \tau}$;
- ▶ Voltando por $e(\tau) = Vc(\tau)$ e $c_k(0) = v_k^T e_0$:

$$e(\tau) = \sum_{k=1}^N (v_k^T e_0) e^{-\lambda_k \tau} v_k, \quad e^{(n)} = \sum_{k=1}^N (v_k^T e_0) (1 - \mu \lambda_k)^n v_k.$$

- ▶ A segunda forma é a contraparte por Euler explícito da primeira: um passo de descenso do gradiente é um passo de Euler sobre o erro.

Com $N_k = \lfloor N_x/2 \rfloor + 1$ modos retidos, $q_1 = \lfloor N_k/4 \rfloor$ e $q_2 = \lfloor N_k/2 \rfloor$:

$$B_{\text{baixa}} = \{ k : 0 \leq k < q_1 \}, \quad B_{\text{média}} = \{ k : q_1 \leq k < q_2 \}, \quad B_{\text{alta}} = \{ k : q_2 \leq k < N_k \}.$$

- ▶ Um quarto do intervalo para cada uma das duas primeiras bandas, metade superior para a última;
- ▶ O erro por banda faz duas médias: sobre os N_t níveis de tempo, depois sobre os modos da banda;
- ▶ A redução alternativa inverte os eixos e colapsa o espectro numa curva contra t :
$$\tilde{E}_{\text{spec}}(t_n) = \frac{1}{N_k} \sum_{k=0}^{N_k-1} E_{\text{spec}}(k, t_n);$$
- ▶ As curvas por banda são registradas a cada 500 iterações, em $\alpha = \beta = 3,21$: métodos pontuais contra a própria solução de treino, operadores contra a amostra mais próxima.

Uma regra única leva toda entrada da rede a $[-1, 1]$,

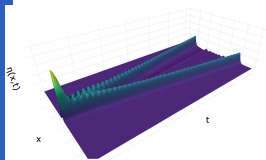
$$\tilde{f} = \frac{2(f - f_{\min})}{f_{\max} - f_{\min}} - 1, \quad f = f_{\min} + \frac{1}{2}(\tilde{f} + 1)(f_{\max} - f_{\min}),$$

a segunda sendo a inversa exata da primeira.

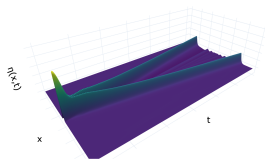
- ▶ **MLP.** Aplica às duas coordenadas; a saída fica em unidades físicas;
- ▶ **PINN.** O mesmo, dentro da passagem direta; a diferenciação automática carrega a regra da cadeia pelo reescalonamento, e o que se impõe é o resíduo do sistema original;
- ▶ **FNO e PINO.** Aplicam aos quatro canais de entrada e aos dois de saída, mais dois canais de coordenada: a camada de elevação lê seis canais;
- ▶ **Unidades de cada termo.** Predição e alvo são comparados normalizados; o PINO devolve os campos a unidades físicas para resíduo e condição inicial.

Painéis de resolução do PINO

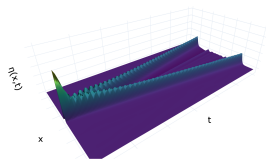
Os dois regimes do PINO fora da grade de treino em $\alpha = \beta = 3,21$, sem retreino.



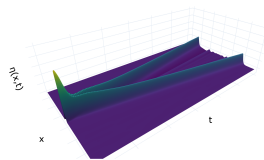
PINO dados, res. 128



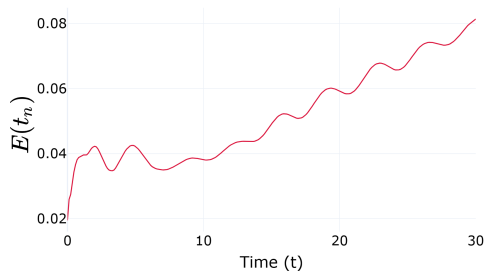
PINO dados, res. 512



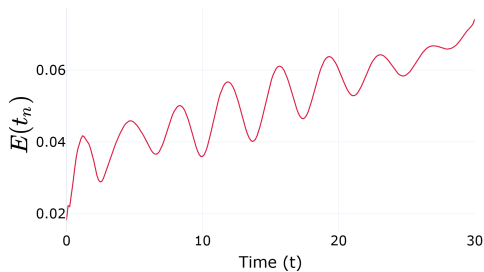
PINO física, res. 128



PINO física, res. 512



PINO (dados e física), erro relativo, res. 256



PINO (física pura), erro relativo, res. 256

O domínio $[-L, L]$ é discretizado em N_x pontos equidistantes $x_j = -L + j \Delta x$. A transformada discreta de Fourier (DFT) e sua inversa são (Trefethen 2000)

$$\hat{f}(k) = \sum_{j=0}^{N_x-1} f(x_j) e^{-2\pi i k j / N_x}, \quad f(x_j) = \frac{1}{N_x} \sum_k \hat{f}(k) e^{2\pi i k j / N_x}.$$

Com o número de onda $\xi_k := \pi k / L$, a derivada vira multiplicação:

$$\widehat{(f_x)}(k) = i \xi_k \hat{f}(k), \quad \widehat{(f_{xx})}(k) = -\xi_k^2 \hat{f}(k).$$

- Cálculo de derivadas muito mais eficiente e confiável;

O domínio $[-L, L]$ é discretizado em N_x pontos equidistantes $x_j = -L + j \Delta x$. A transformada discreta de Fourier (DFT) e sua inversa são (Trefethen 2000)

$$\hat{f}(k) = \sum_{j=0}^{N_x-1} f(x_j) e^{-2\pi i k j / N_x}, \quad f(x_j) = \frac{1}{N_x} \sum_k \hat{f}(k) e^{2\pi i k j / N_x}.$$

Com o número de onda $\xi_k := \pi k / L$, a derivada vira multiplicação:

$$\widehat{(f_x)}(k) = i \xi_k \hat{f}(k), \quad \widehat{(f_{xx})}(k) = -\xi_k^2 \hat{f}(k).$$

- ▶ Cálculo de derivadas muito mais eficiente e confiável;
- ▶ Cada DFT custa $O(N_x \log N_x)$ pela transformada rápida de Fourier (FFT) (Cooley e Tukey 1965).

Aplicamos a DFT a cada equação, usando $\widehat{(f_x)} = i \xi_k \hat{f}$, temos $\widehat{(u_{xxt})} = -\xi_k^2 \hat{u}_t$ e também um dos termos não lineares pode ser escrito como $uu_x = \frac{1}{2}(u^2)_x$ (Martins 2023):

$$\eta_t + u_x + \alpha (\eta u)_x = 0 \quad \longrightarrow \quad \hat{\eta}_t + i \xi_k \hat{u} + \alpha i \xi_k \widehat{(\eta u)} = 0,$$

$$u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \frac{\alpha}{2} (u^2)_x = 0 \quad \longrightarrow \quad \hat{u}_t + \frac{\beta}{3} \xi_k^2 \hat{u}_t + i \xi_k \hat{\eta} + \alpha i \xi_k \widehat{\left(\frac{1}{2} u^2\right)} = 0.$$

Aplicamos a DFT a cada equação, usando $\widehat{(f_x)} = i \xi_k \hat{f}$, temos $\widehat{(u_{xxt})} = -\xi_k^2 \hat{u}_t$ e também um dos termos não lineares pode ser escrito como $uu_x = \frac{1}{2}(u^2)_x$ (Martins 2023):

$$\eta_t + u_x + \alpha (\eta u)_x = 0 \quad \longrightarrow \quad \hat{\eta}_t + i \xi_k \hat{u} + \alpha i \xi_k \widehat{(\eta u)} = 0,$$

$$u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \frac{\alpha}{2} (u^2)_x = 0 \quad \longrightarrow \quad \hat{u}_t + \frac{\beta}{3} \xi_k^2 \hat{u}_t + i \xi_k \hat{\eta} + \alpha i \xi_k \widehat{\left(\frac{1}{2} u^2\right)} = 0.$$

Na segunda equação, agrupamos os termos em $\hat{u}_t$:

$$\left(1 + \frac{\beta \xi_k^2}{3}\right) \hat{u}_t = -i \xi_k \hat{\eta} - \alpha i \xi_k \widehat{\left(\frac{1}{2} u^2\right)}.$$

Aplicamos a DFT a cada equação, usando $\widehat{(f_x)} = i \xi_k \hat{f}$, temos $\widehat{(u_{xxt})} = -\xi_k^2 \hat{u}_t$ e também um dos termos não lineares pode ser escrito como $uu_x = \frac{1}{2}(u^2)_x$ (Martins 2023):

$$\eta_t + u_x + \alpha (\eta u)_x = 0 \quad \longrightarrow \quad \hat{\eta}_t + i \xi_k \hat{u} + \alpha i \xi_k \widehat{(\eta u)} = 0,$$

$$u_t - \frac{\beta}{3} u_{xxt} + \eta_x + \frac{\alpha}{2} (u^2)_x = 0 \quad \longrightarrow \quad \hat{u}_t + \frac{\beta}{3} \xi_k^2 \hat{u}_t + i \xi_k \hat{\eta} + \alpha i \xi_k \widehat{\left(\frac{1}{2} u^2\right)} = 0.$$

Na segunda equação, agrupamos os termos em $\hat{u}_t$:

$$\left(1 + \frac{\beta \xi_k^2}{3}\right) \hat{u}_t = -i \xi_k \hat{\eta} - \alpha i \xi_k \widehat{\left(\frac{1}{2} u^2\right)}.$$

Isolar $\hat{u}_t$ vira uma divisão por modo; a primeira equação já está resolvida para $\hat{\eta}_t$.

Sob a DFT, o sistema de Boussinesq vira, para cada número de onda k , uma Equação Diferencial Ordinária (EDO) em t (Martins 2023):

$$\begin{cases} \hat{\eta}_t(k, t) = -i \xi_k \hat{u}(k, t) - \alpha i \xi_k \widehat{(\eta u)}(k, t), \\ \hat{u}_t(k, t) = \frac{-i \xi_k \hat{\eta}(k, t) - \alpha i \xi_k \widehat{(\frac{1}{2} u^2)}(k, t)}{1 + \frac{\beta \xi_k^2}{3}}. \end{cases}$$

Tomando $\hat{Y}(k, t) = (\hat{\eta}, \hat{u})$, o problema toma a forma $\hat{Y}_t = F(\hat{Y})$.

Assim, chegamos a um sistema de EDOs para cada índice de frequência k .

Com a discretização em N_x pontos, são N_x sistemas de EDOs a serem evoluídos no tempo.

Para evoluir $\hat{Y}_t = F(\hat{Y})$ no tempo, usamos Runge–Kutta de quarta ordem (Süli e Mayers 2003; Burden e Faires 2011):

$$\begin{aligned}\kappa_1 &= F(\hat{Y}_n), & \kappa_2 &= F\left(\hat{Y}_n + \frac{\Delta t}{2} \kappa_1\right), \\ \kappa_3 &= F\left(\hat{Y}_n + \frac{\Delta t}{2} \kappa_2\right), & \kappa_4 &= F\left(\hat{Y}_n + \Delta t \kappa_3\right),\end{aligned}$$

$$\hat{Y}_{n+1} = \hat{Y}_n + \frac{\Delta t}{6} (\kappa_1 + 2\kappa_2 + 2\kappa_3 + \kappa_4).$$

- Cada estágio volta ao espaço físico para os produtos não lineares e retorna ao de Fourier para as derivadas;

Para evoluir $\hat{Y}_t = F(\hat{Y})$ no tempo, usamos Runge–Kutta de quarta ordem (Süli e Mayers 2003; Burden e Faires 2011):

$$\begin{aligned}\kappa_1 &= F(\hat{Y}_n), & \kappa_2 &= F\left(\hat{Y}_n + \frac{\Delta t}{2} \kappa_1\right), \\ \kappa_3 &= F\left(\hat{Y}_n + \frac{\Delta t}{2} \kappa_2\right), & \kappa_4 &= F\left(\hat{Y}_n + \Delta t \kappa_3\right),\end{aligned}$$

$$\hat{Y}_{n+1} = \hat{Y}_n + \frac{\Delta t}{6} (\kappa_1 + 2\kappa_2 + 2\kappa_3 + \kappa_4).$$

- ▶ Cada estágio volta ao espaço físico para os produtos não lineares e retorna ao de Fourier para as derivadas;
- ▶ Acurácia espectral no espaço e quarta ordem no tempo: referência de alta acurácia para todos os métodos treinados (Trefethen 2000; Steinmoeller et al. 2012).

- Burden, R. L. e J. D. Faires (2011). *Numerical Analysis*. 9^a ed. Boston: Brooks/Cole.
- Cooley, J. W. e J. W. Tukey (1965). “An algorithm for the machine calculation of complex Fourier series”. Em: *Mathematics of Computation* 19.90, pp. 297–301.
- Cybenko, G. (1989). “Approximation by superpositions of a sigmoidal function”. Em: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314.
- Golub, G. H. e C. F. Van Loan (2013). *Matrix Computations*. 4^a ed. Baltimore: Johns Hopkins University Press.
- Hornik, K., M. Stinchcombe e H. White (1989). “Multilayer feedforward networks are universal approximators”. Em: *Neural Networks* 2.5, pp. 359–366.
- LeCun, Y., Y. Bengio e G. Hinton (2015). “Deep learning”. Em: *Nature* 521.7553, pp. 436–444.
- Martins, L. G. (2023). “Estudo Numérico do Modelo Boussinesq com Topografia e Pressão Variável no Espaço e no Tempo”. Diss. de mestr. Curitiba: Universidade Federal do Paraná.
- McCulloch, W. S. e W. Pitts (1943). “A logical calculus of the ideas immanent in nervous activity”. Em: *Bulletin of Mathematical Biophysics* 5.4, pp. 115–133.

- Minsky, M. L. (1954). “Theory of Neural-Analog Reinforcement Systems and Its Application to the Brain-Model Problem”. Tese de dout. Princeton University.
- Rosenblatt, F. (1958). “The perceptron: a probabilistic model for information storage and organization in the brain”. Em: *Psychological Review* 65.6, pp. 386–408.
- Rumelhart, D. E., G. E. Hinton e R. J. Williams (1986). “Learning representations by back-propagating errors”. Em: *Nature* 323.6088, pp. 533–536.
- Steinmoeller, D. T., M. Stastna e K. G. Lamb (2012). “Fourier pseudospectral methods for 2D Boussinesq-type equations”. Em: *Ocean Modelling* 52–53, pp. 76–89.
- Strang, G. (2006). *Linear Algebra and Its Applications*. 4^a ed. Belmont: Thomson Brooks/Cole.
- Süli, E. e D. F. Mayers (2003). *An Introduction to Numerical Analysis*. Cambridge: Cambridge University Press.
- Trefethen, L. N. (2000). *Spectral Methods in MATLAB*. Philadelphia: Society for Industrial e Applied Mathematics.